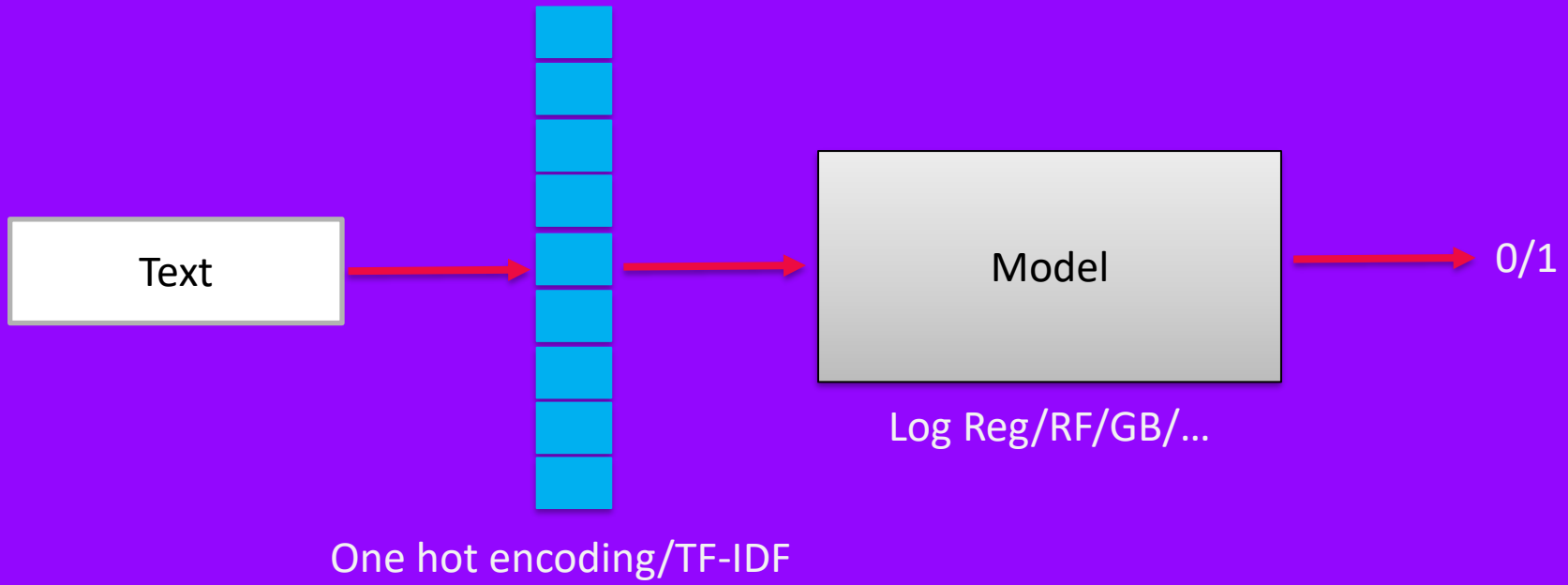
ИТМО

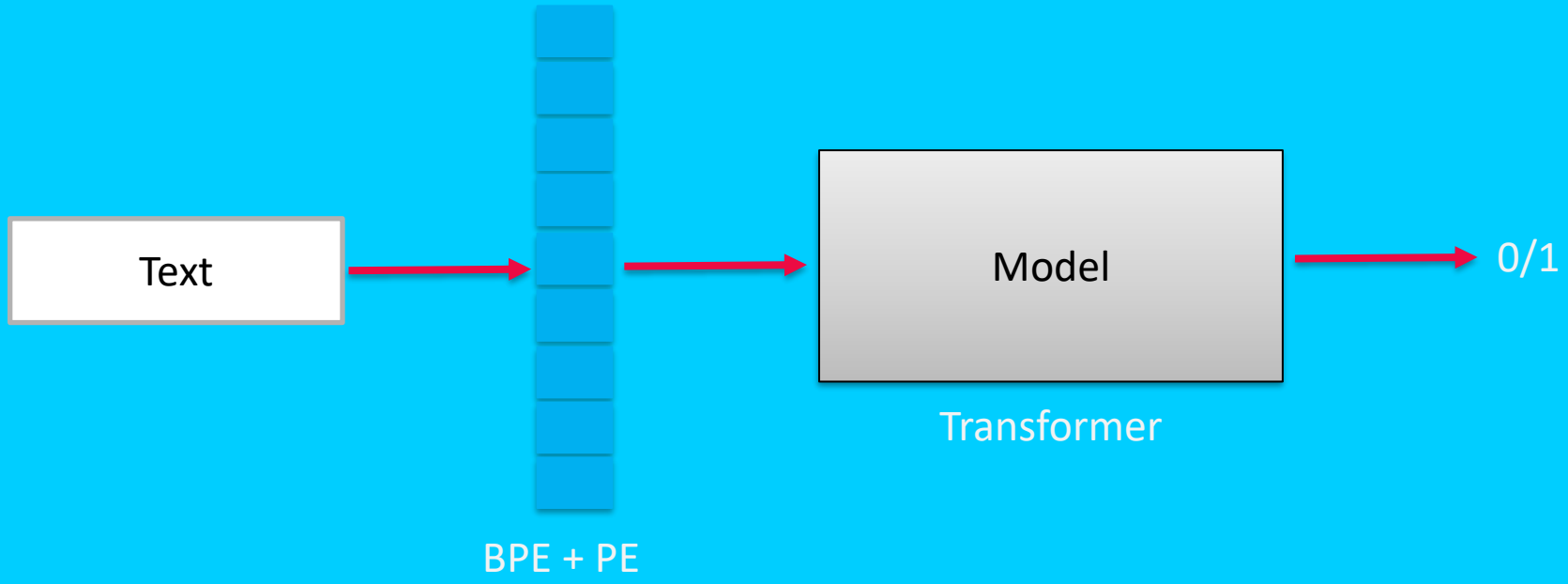
Transformers: Basics

Современные архитектуры нейронных сетей

Классификация текста: классика



Классификация текста: Transformer



Как векторизуется датасет текста?

$$X \in \mathbb{R}^{n \times k \times d}$$

n – размер датасета
d – размер вектора
k – количество токенов
X – датасет

$$x_i \sim X$$

x_i -- текст из датасета

Как векторизовать текст?

Вариант 1: Сделать эмбединг для каждого слова/предложения/…

Слишком большое разнообразие (k -> ∞)

Вариант 2: Сделать эмбединг для каждого символа

Для больший текстов k*d -> ∞

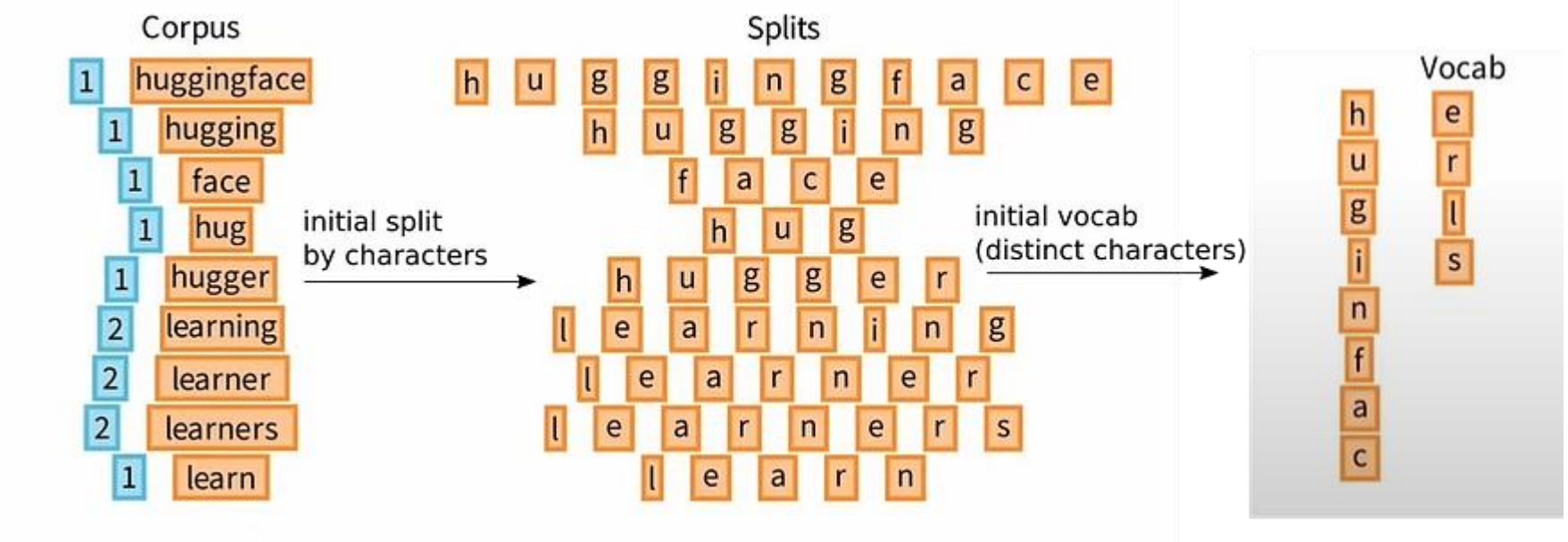
Вариант 3: Сделать эмбединг повторяющихся символов

Оптимальный выбор k

BPE: Byte-Pair Encoding

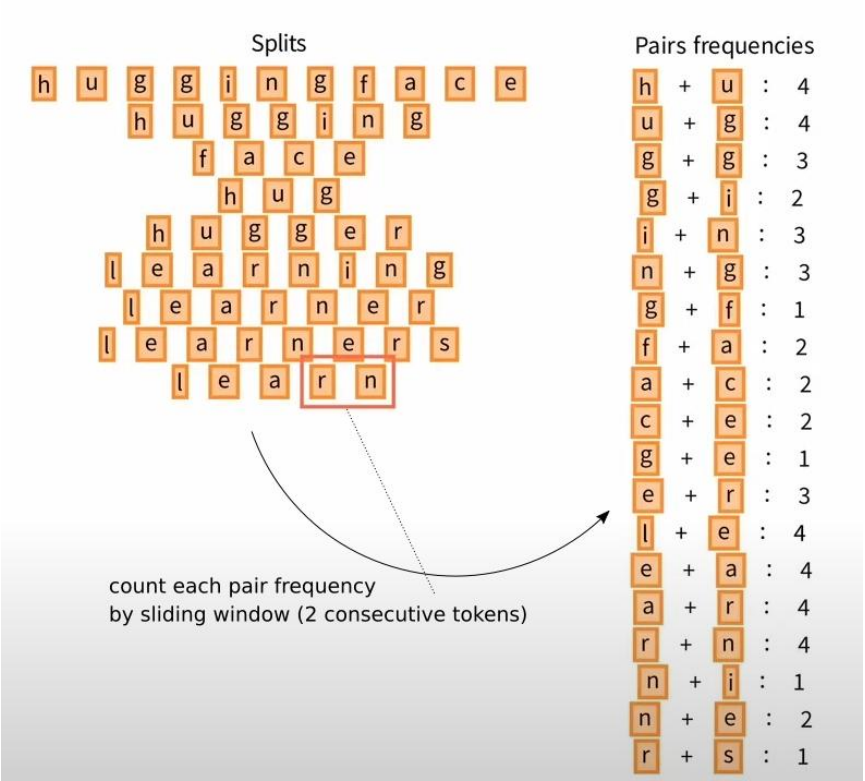
ИТМО

Шаг 1: Инициализация словаря по исходному датасету



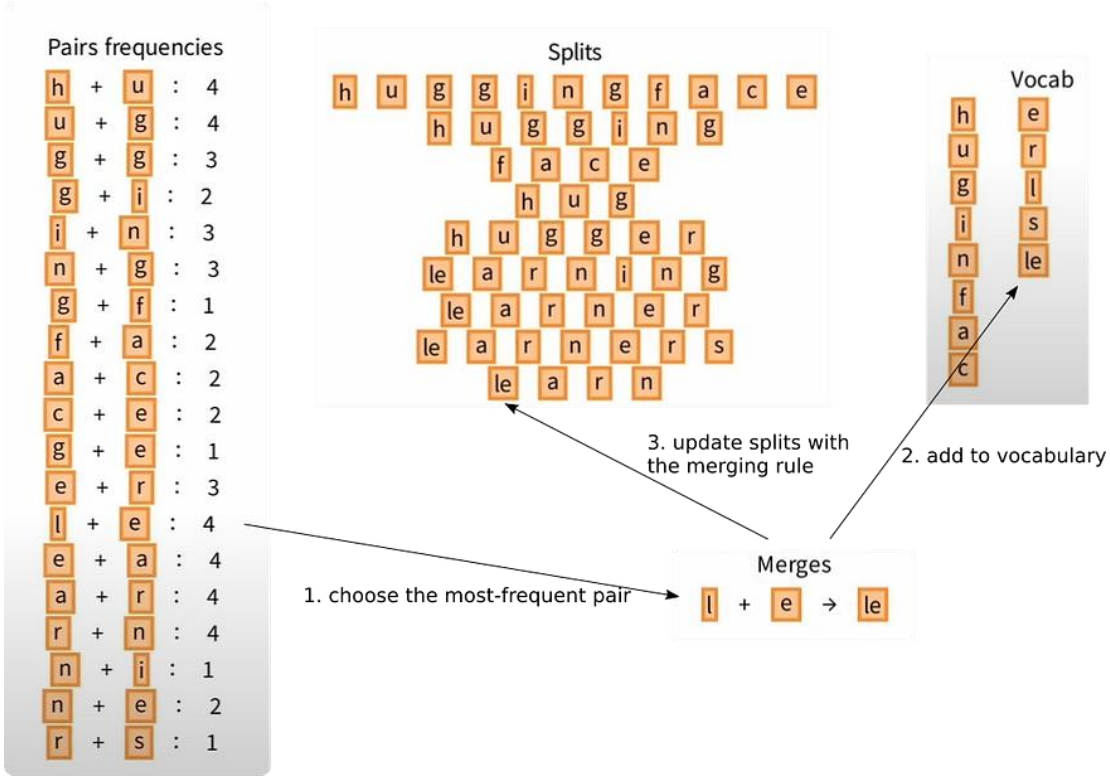
BPE: Byte-Pair Encoding

Шаг 2: Объединение пар символов



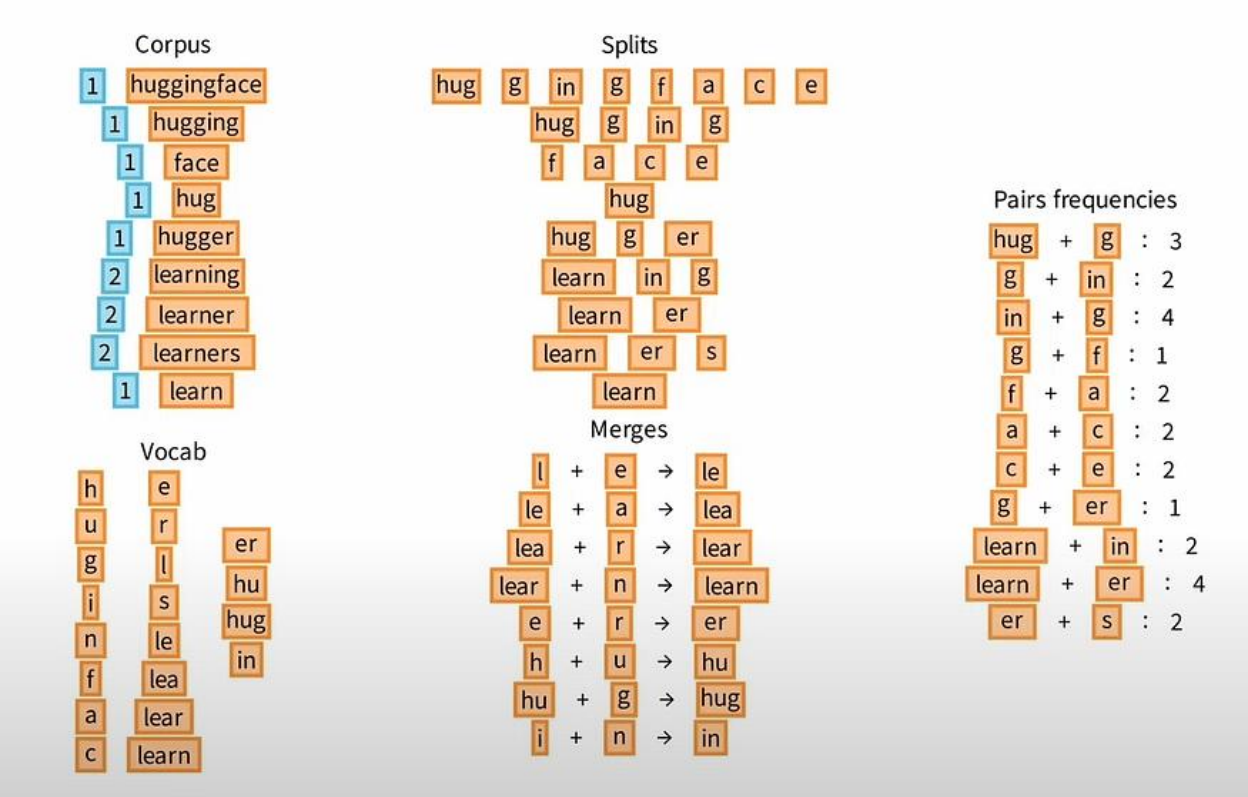
BPE: Byte-Pair Encoding

Шаг 3: Добавление новой комбинации в словарь

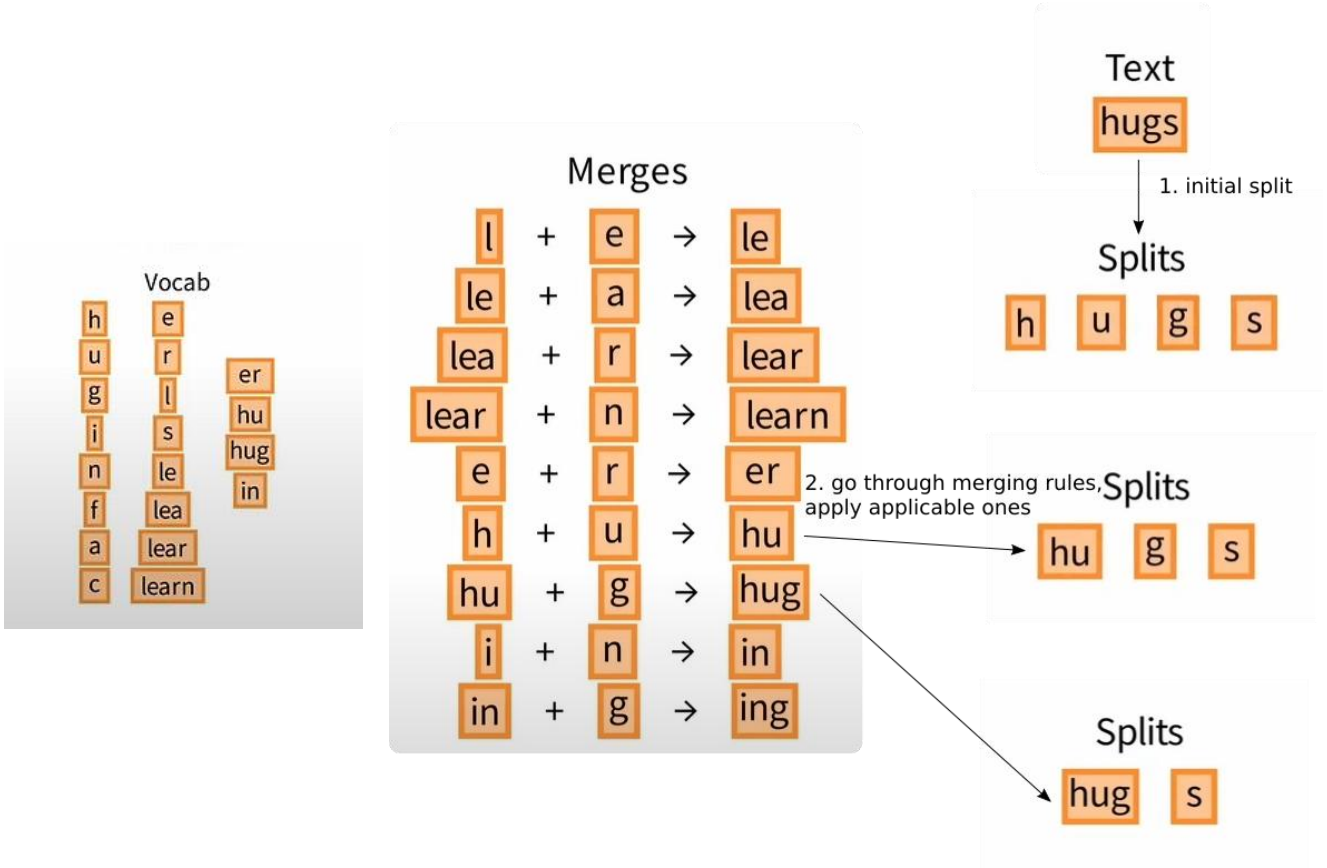


BPE: Byte-Pair Encoding

Шаг 4: Получение финального словаря токенов



BPE: Byte-Pair Encoding (Inference)



len(Vocab) = k

Как выбрать размер d?

$$X \in \mathbb{R}^{n \times k \times d}$$

n – размер датасета
d – размер вектора
k – количество токенов
X – датасет

OhE/ Tf-idf: d = 1

d – гиперамарамет архитектуры

Маленькое d→ меньше параметров, но хуже способность модели улавливать сложные зависимости.

Большое d→ больше параметров, лучше качество, но выше вычислительные затраты.

Как выбрать размер d?

Типичные значения d

BERT-base: $d = 768$
BERT-large: $d = 1024$
GPT-2 small: $d = 768$
GPT-3: $d = 12{,}288$

Каждый токен обучается во время обучения!

Позиционирование токенов

Проблема: Как объяснить нейронке, что нужно обрабатывать токены слева направо и что позиционирование важно в тексте?

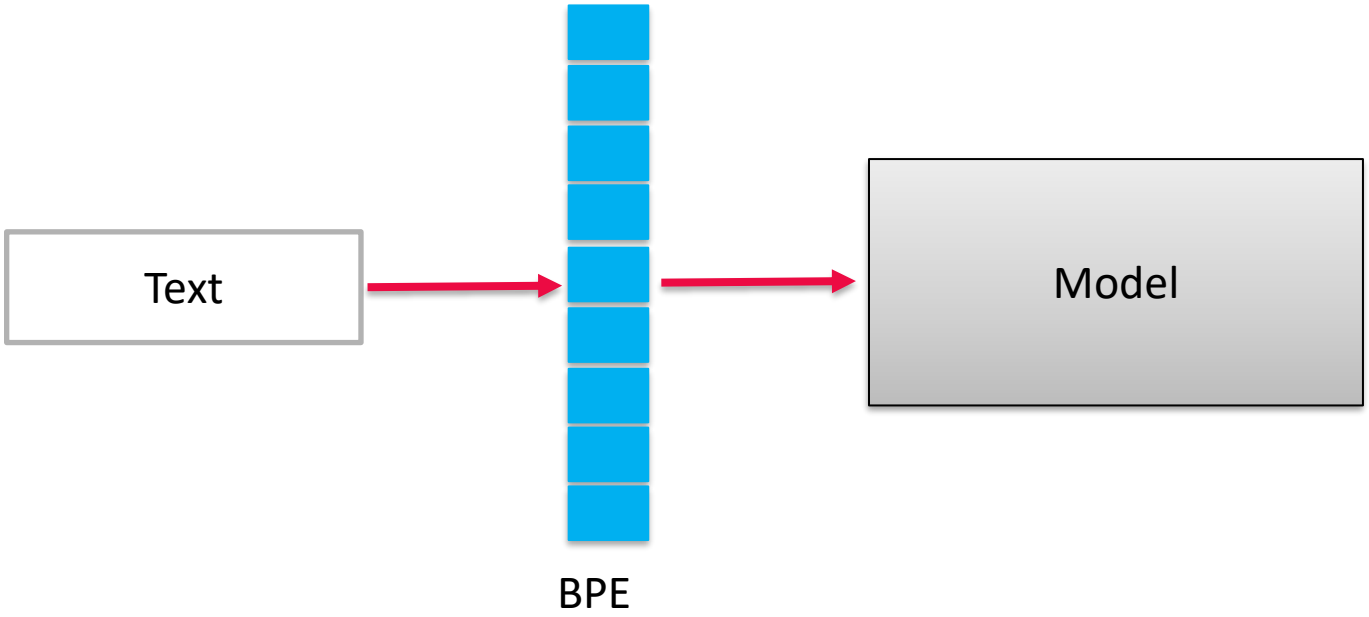
Пример:

"Кошка поймала мышь.“ –> один набор токенов

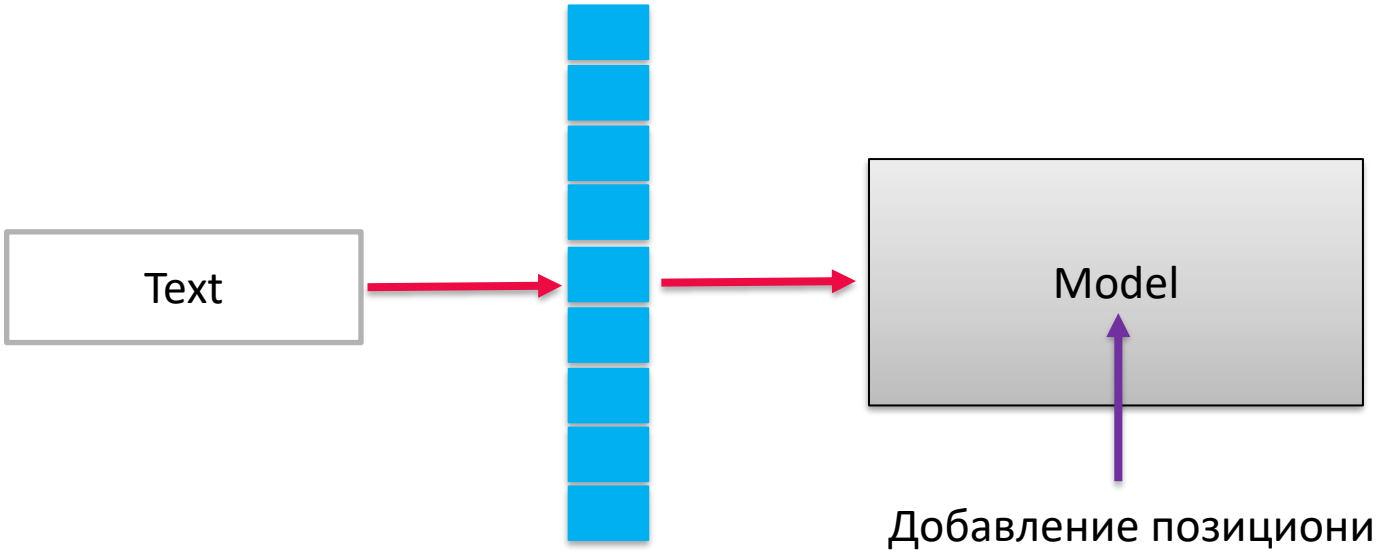
Или

"Мышь поймала кошку." –> такой же набор токенов, но смысл другой

Позиционирование токенов

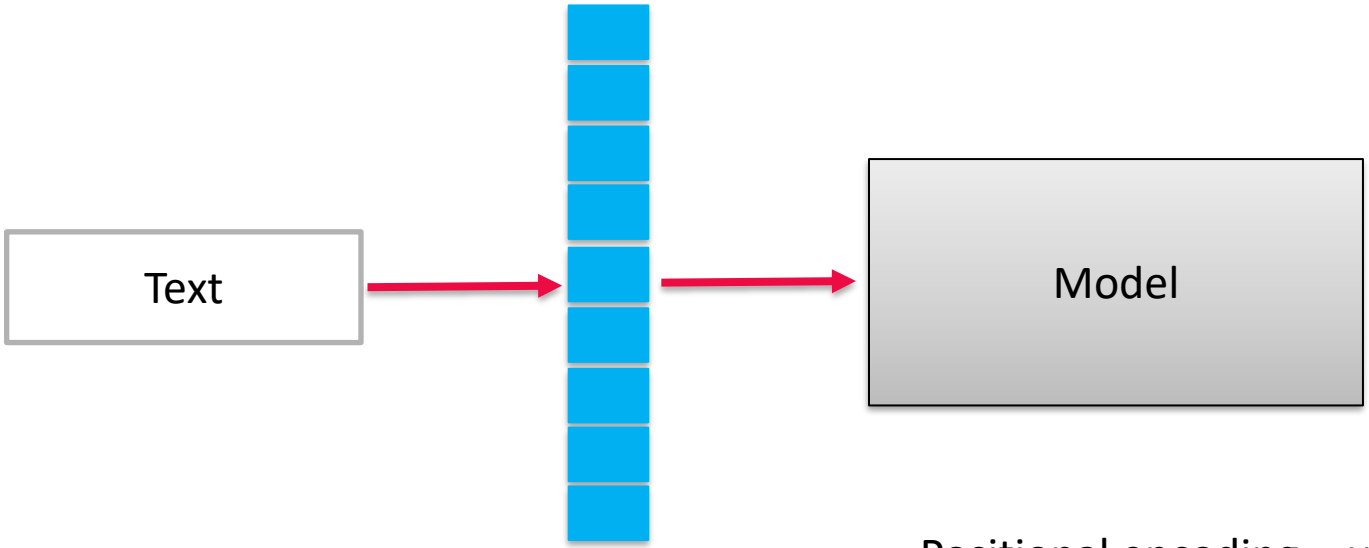


Позиционирование токенов

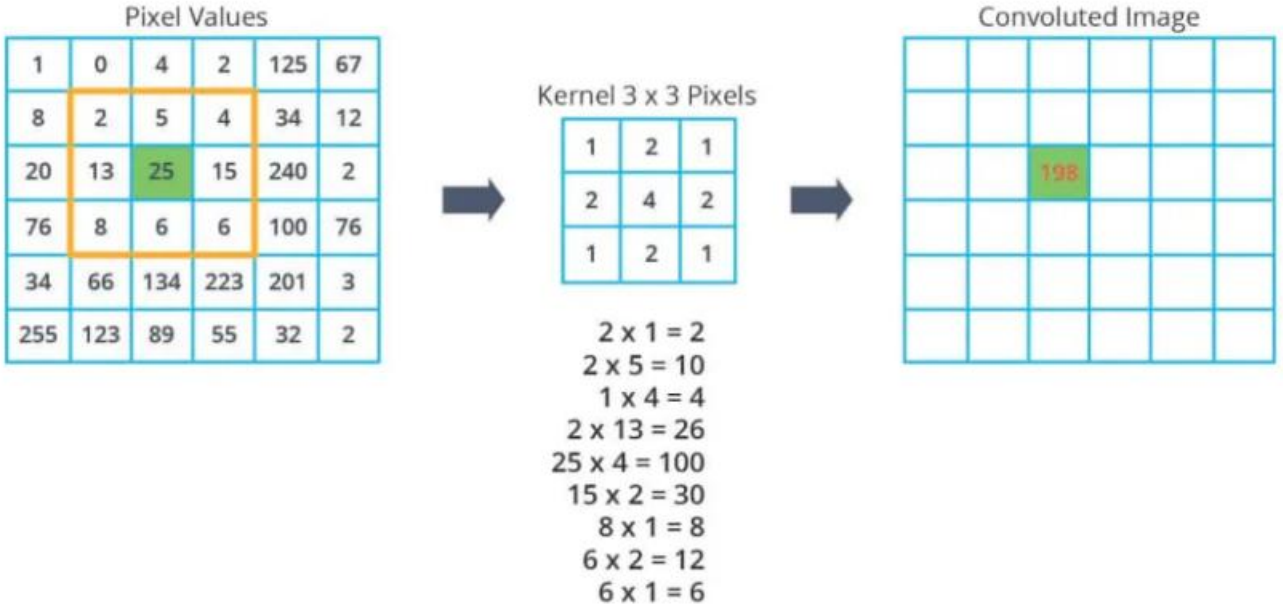


Добавление позиционирования в модель показало не самые эффективные результаты

Позиционирование токенов



Как работает свертка?

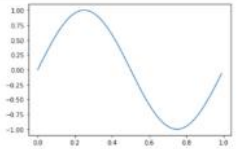
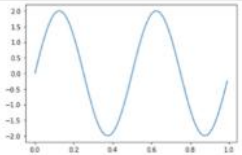
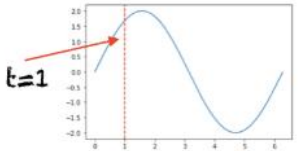


Positional encoding

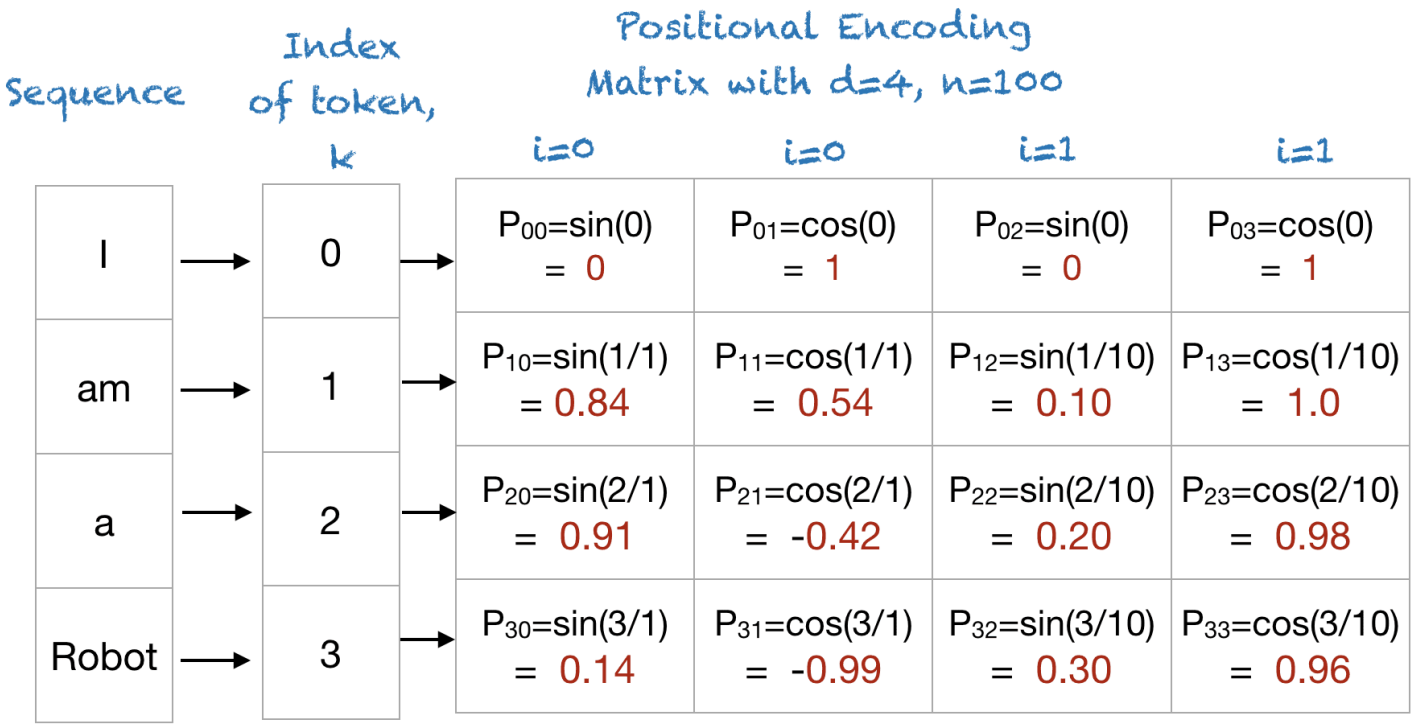
$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right)$$
$$P(k, 2i+1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

k – позиция токена в 0 ≤ k ≤ L (где L – это всего токенов в x)
d – размер вектора
$P(k,j)$ – добавление значение позиции для k-ого токена в j-ый элемент вектора размерности
n – Некоторый скаляр (обычно равен 10.000)

Positional encoding: Интуиция

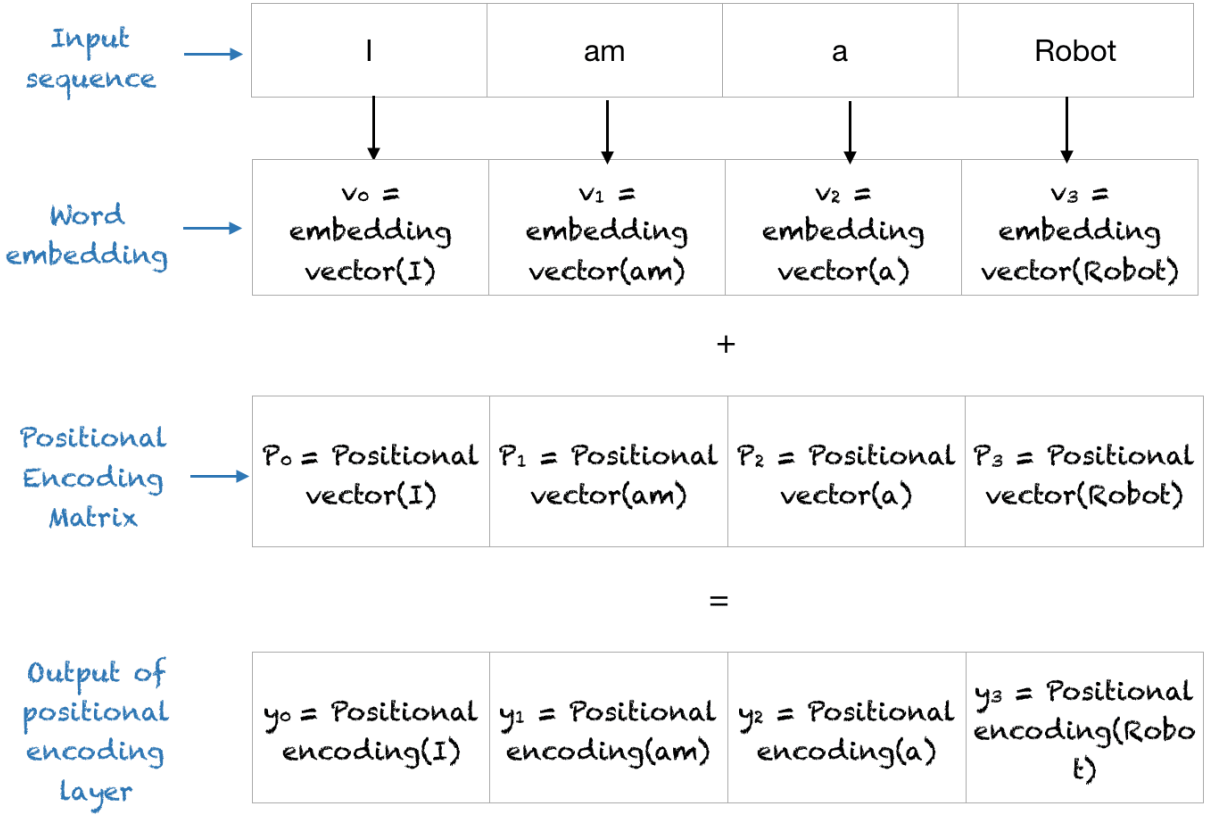
Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

Positional encoding: Пример

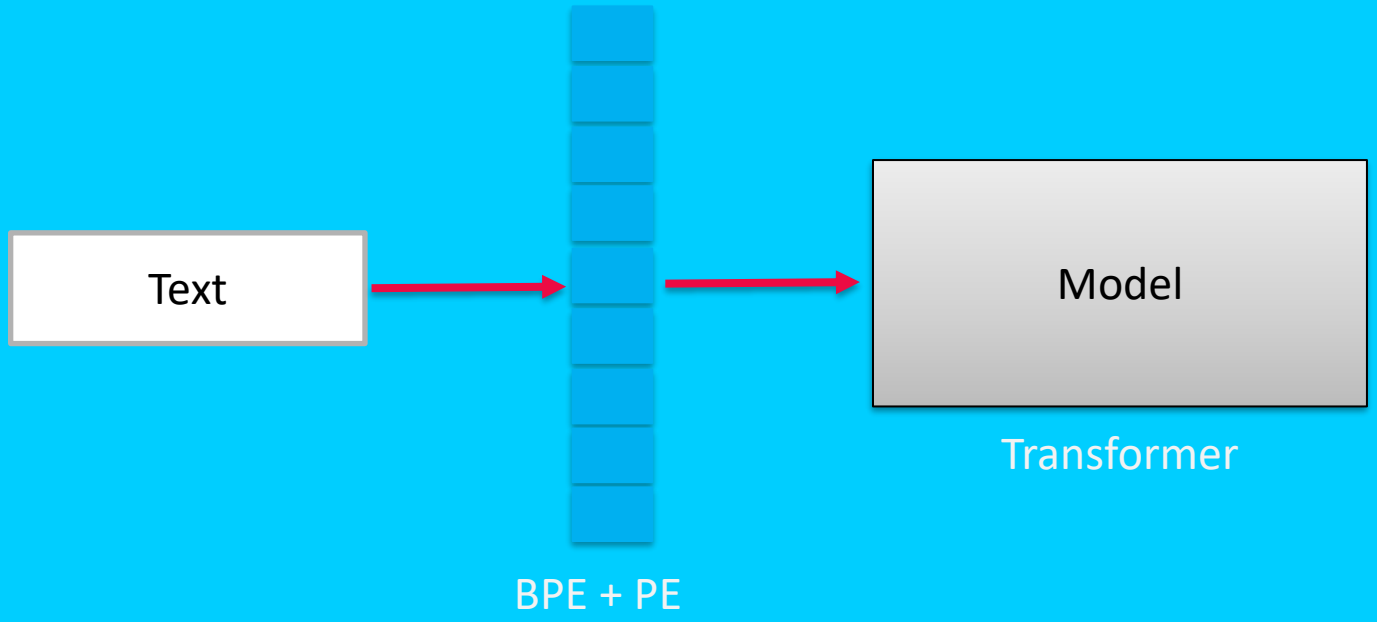


Positional Encoding Matrix for the sequence 'I am a robot'

BPE + PE

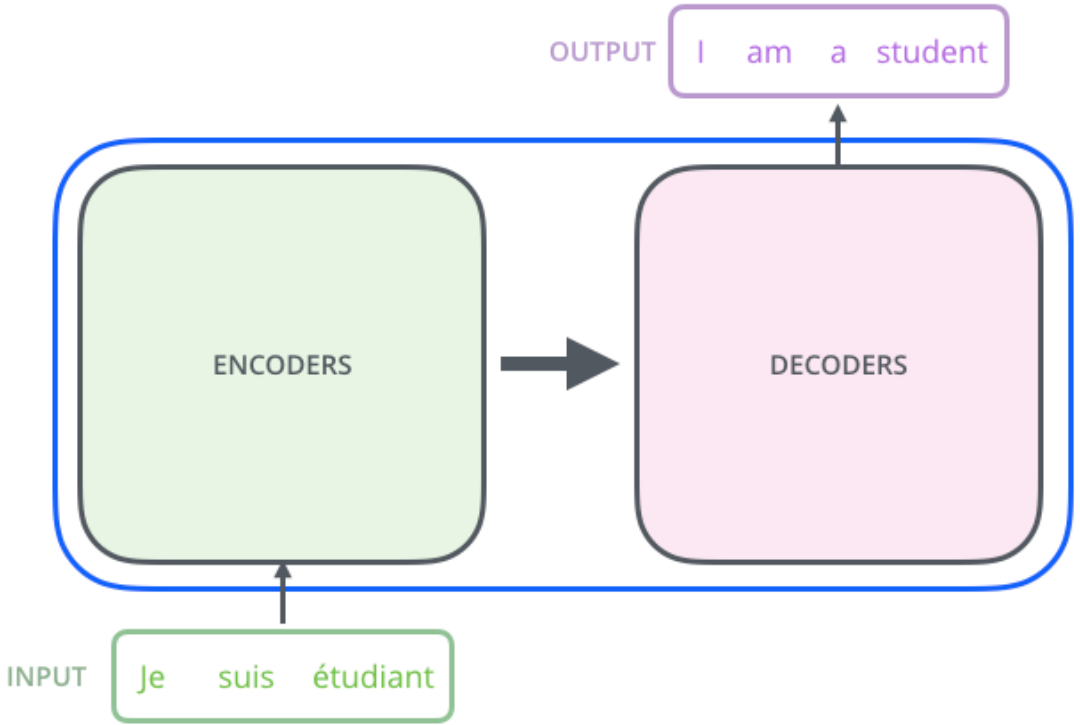

Input sequence
I
am
a
Robot
Word embedding
v0 = embedding vector(I)
v1 = embedding vector(am)
v2 = embedding vector(a)
v3 = embedding vector(Robot)
+
Positional Encoding Matrix
P0 = Positional vector(I)
P1 = Positional vector(am)
P2 = Positional vector(a)
P3 = Positional vector(Robot)
=
Output of positional encoding layer
y0 = Positional encoding(I)
y1 = Positional encoding(am)
y2 = Positional encoding(a)
y3 = Positional encoding(Robot)

Классификация текста: Transformer

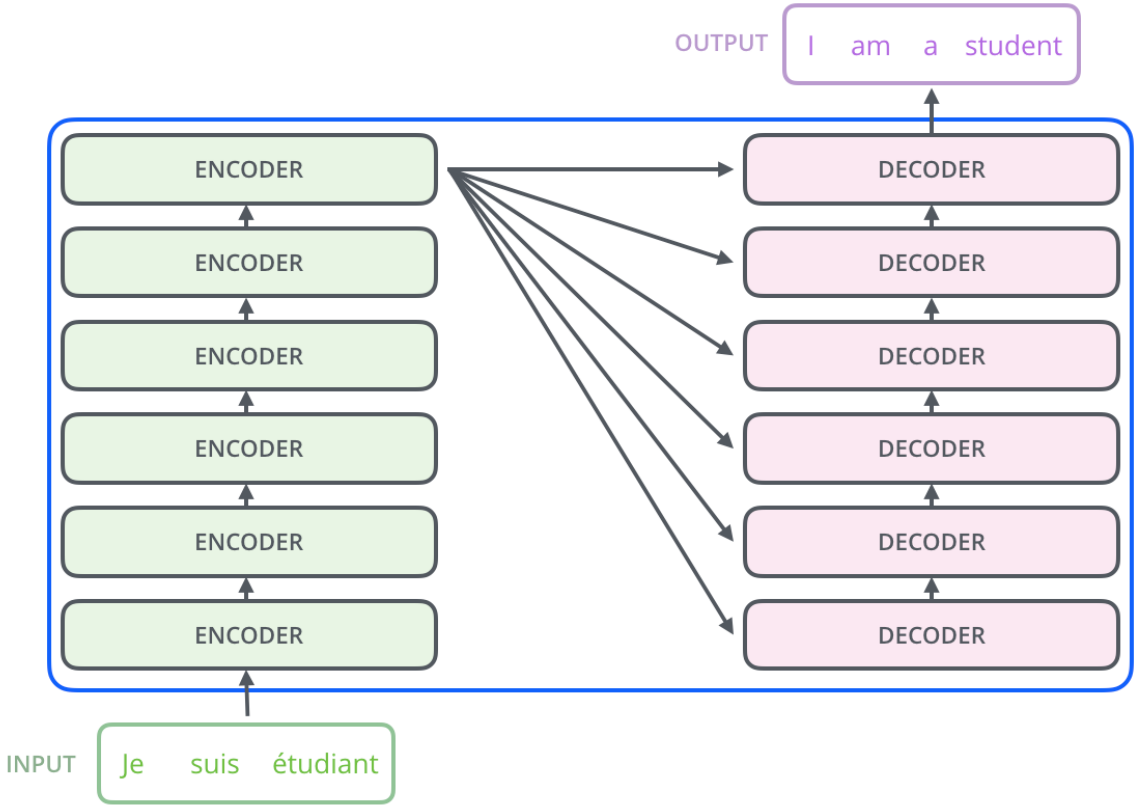


Transformer

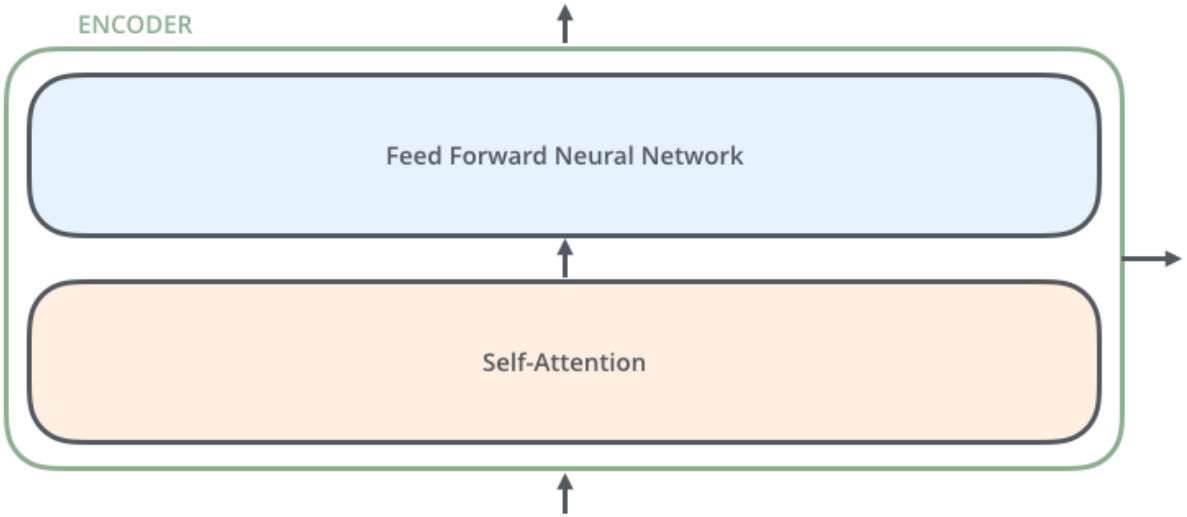
Transformer



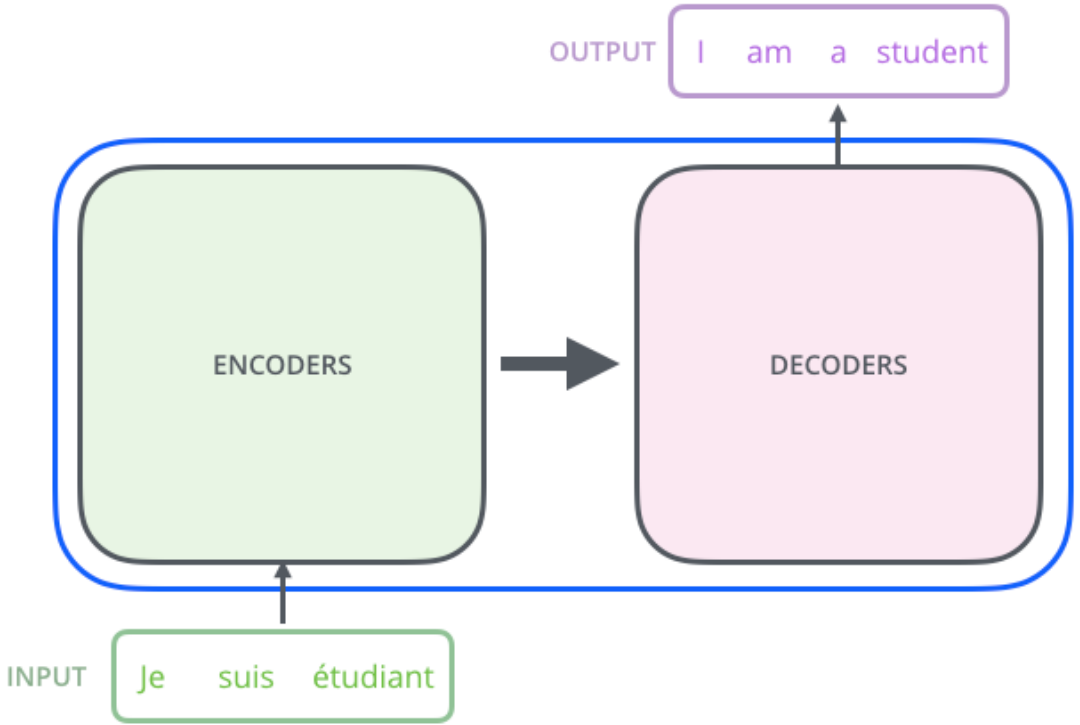
Transformer



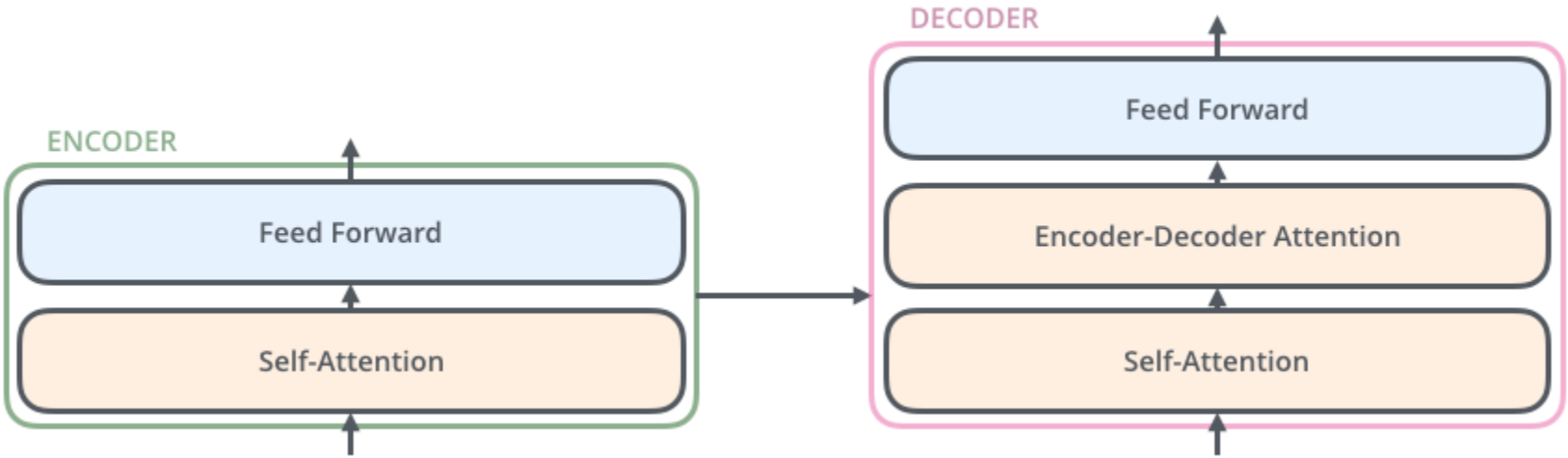
Transformer: Encoder



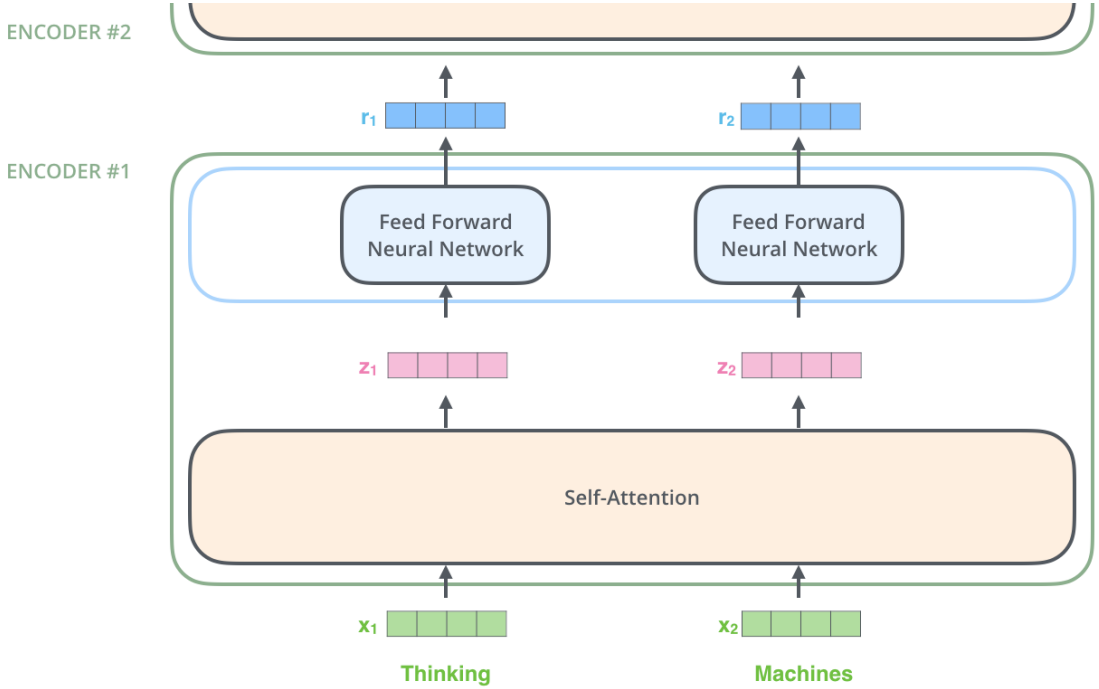
Transformer

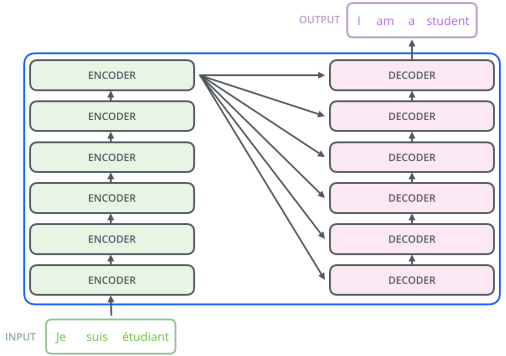


Transformer: Decoder



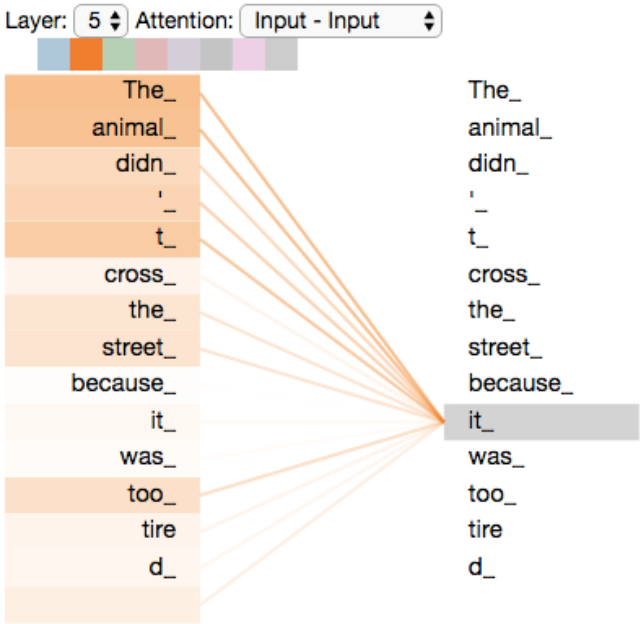
Encoder to Encoder





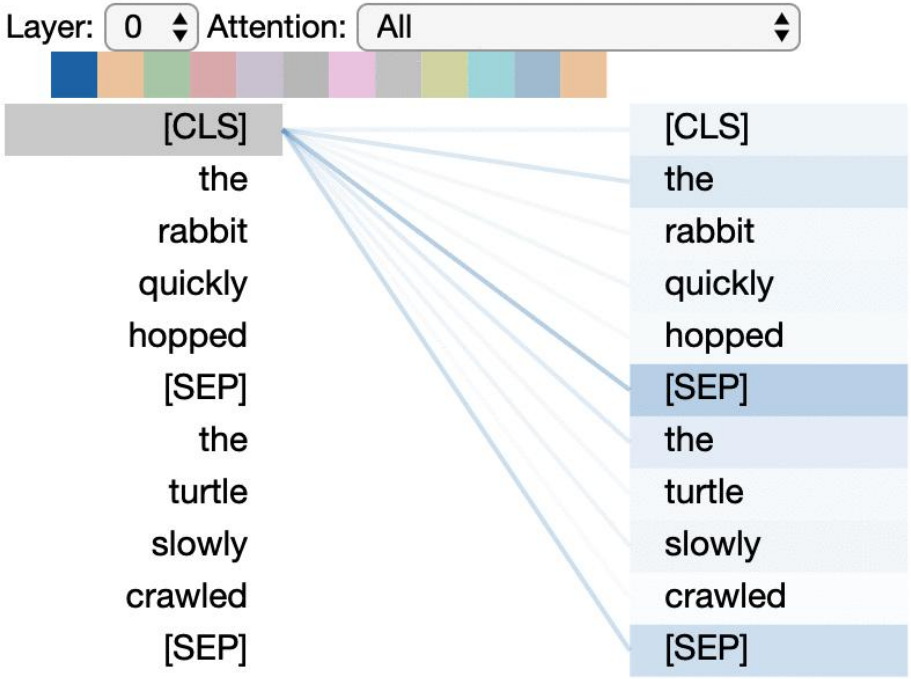
Self-Attention: Интуиция

”The animal didn't cross the street because it was too tired”

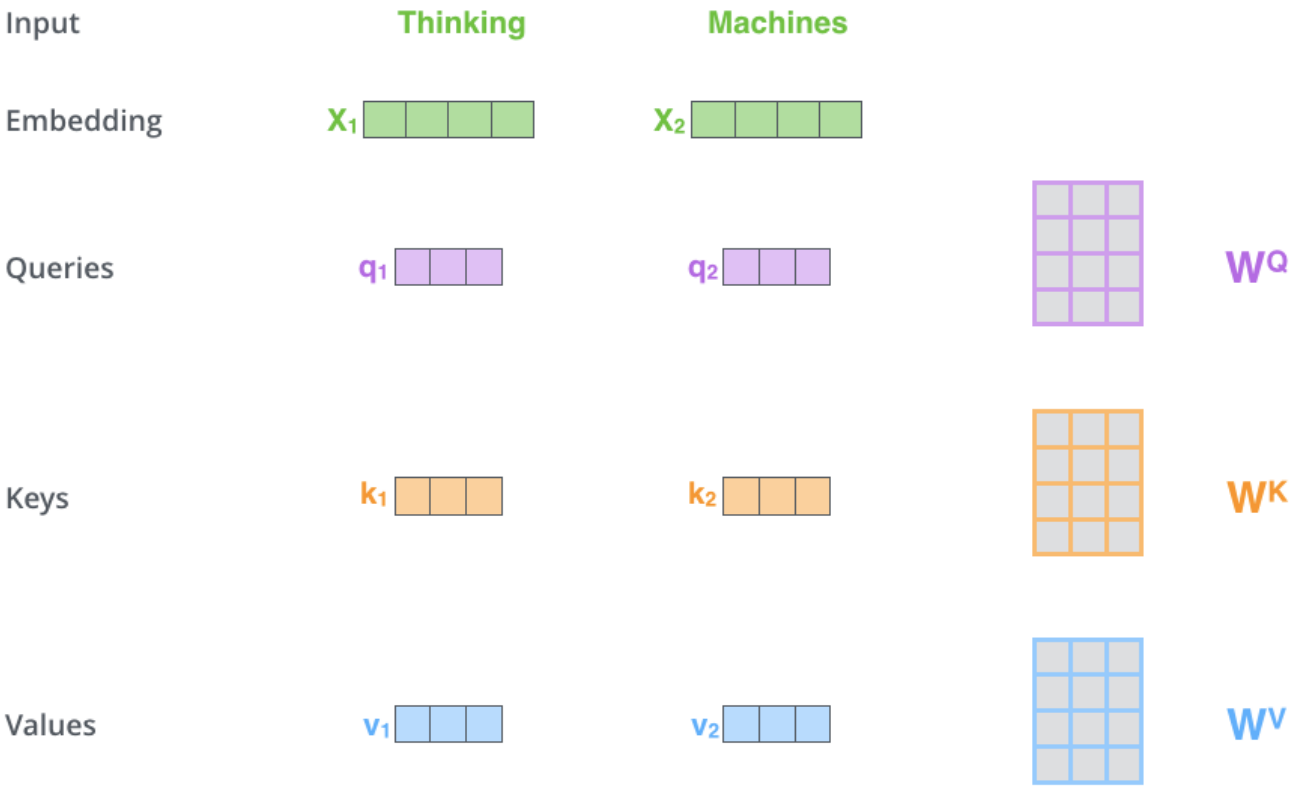


Self-Attention вычисляет для каждого токена его связи с другими токенами во входной последовательности, чтобы выявить контекстные зависимости и точнее закодировать значение слова.

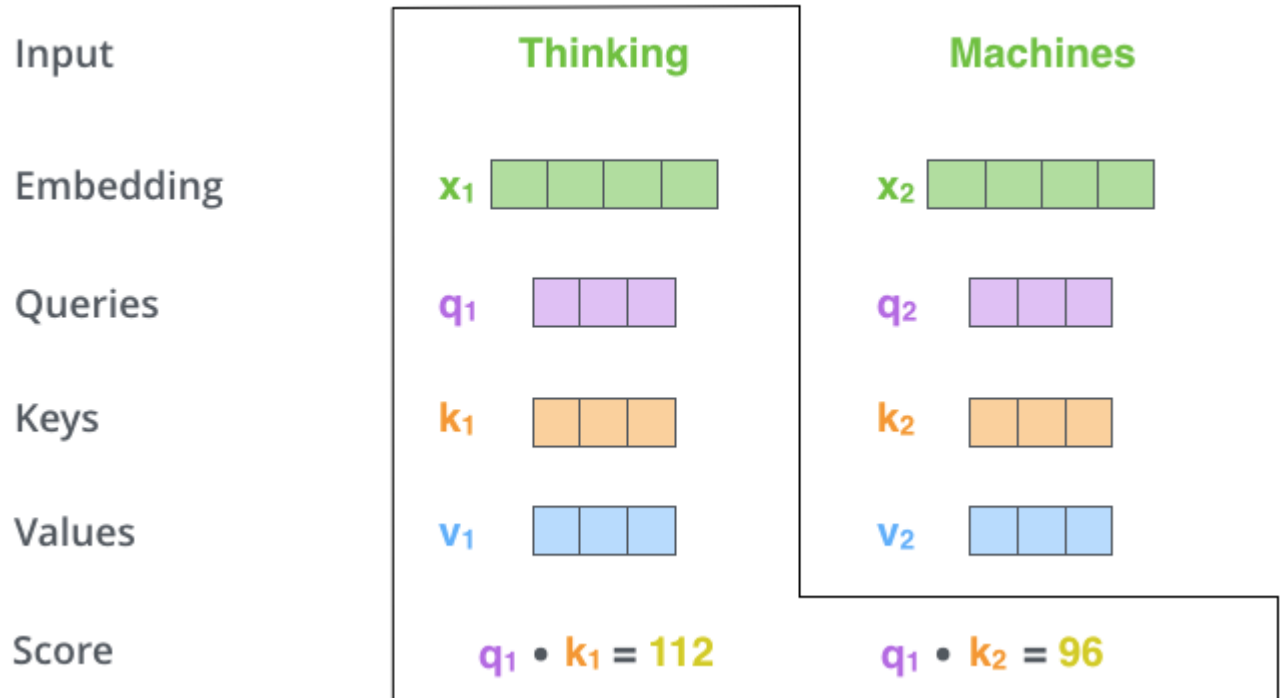
Self-Attention: Интуиция



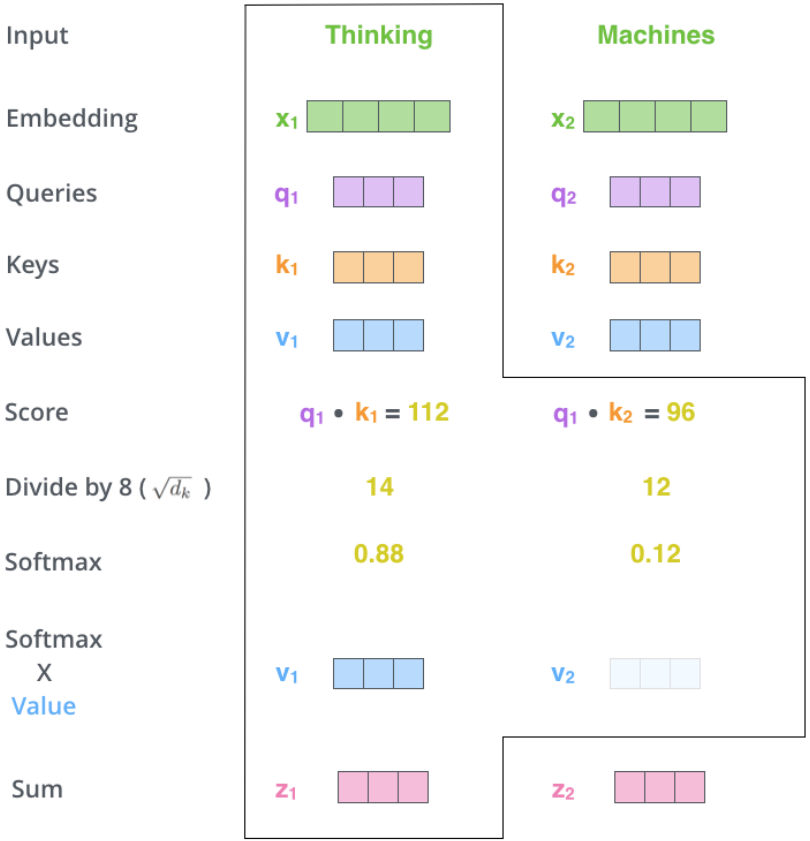
Self-Attention: Details



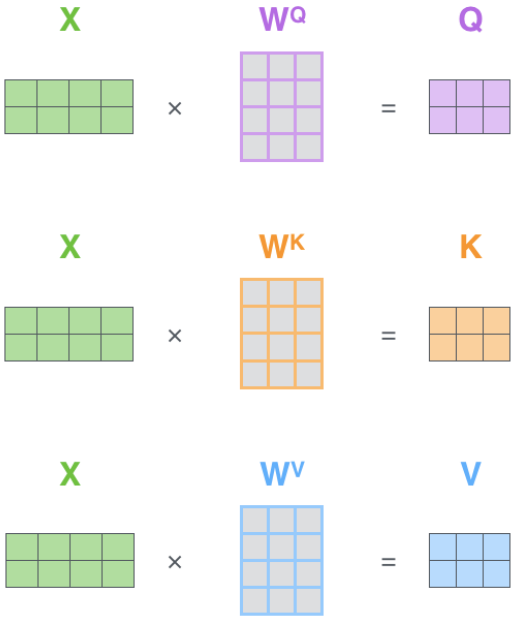
Self-Attention: Details

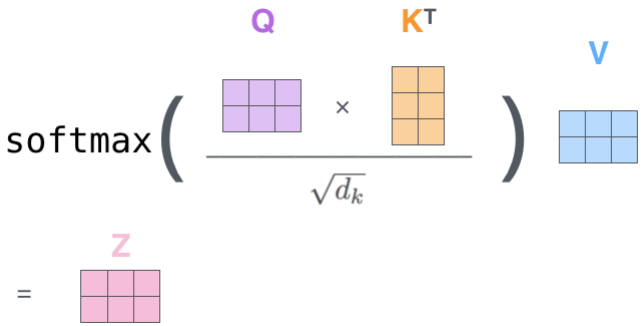


Self-Attention: Details



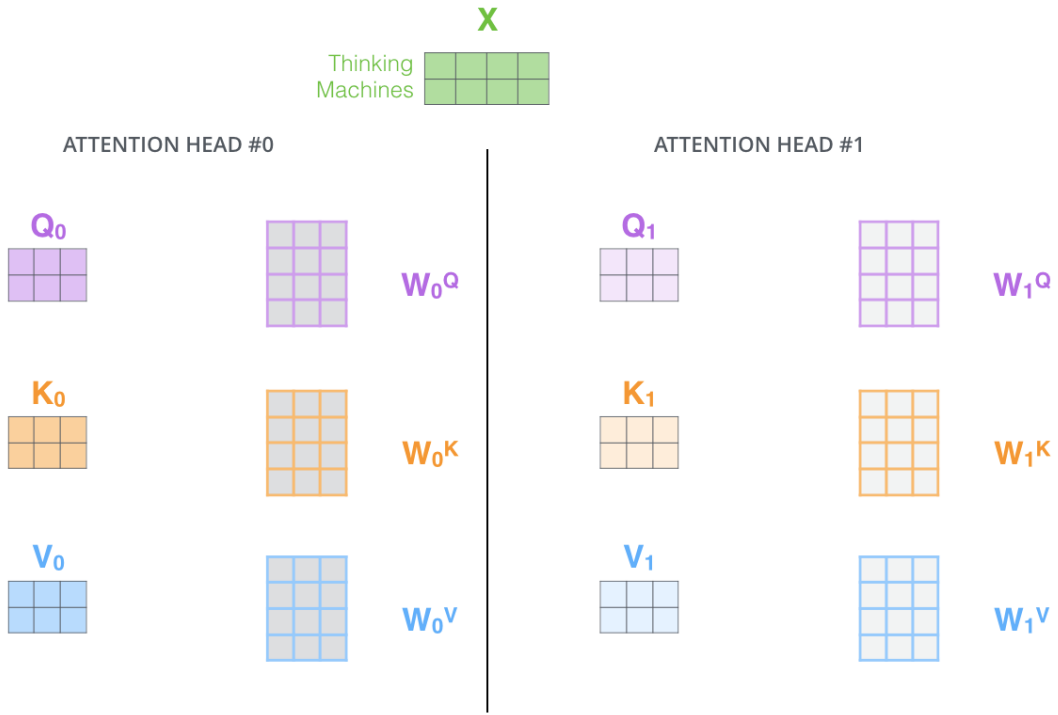
Self-Attention: Matrix form



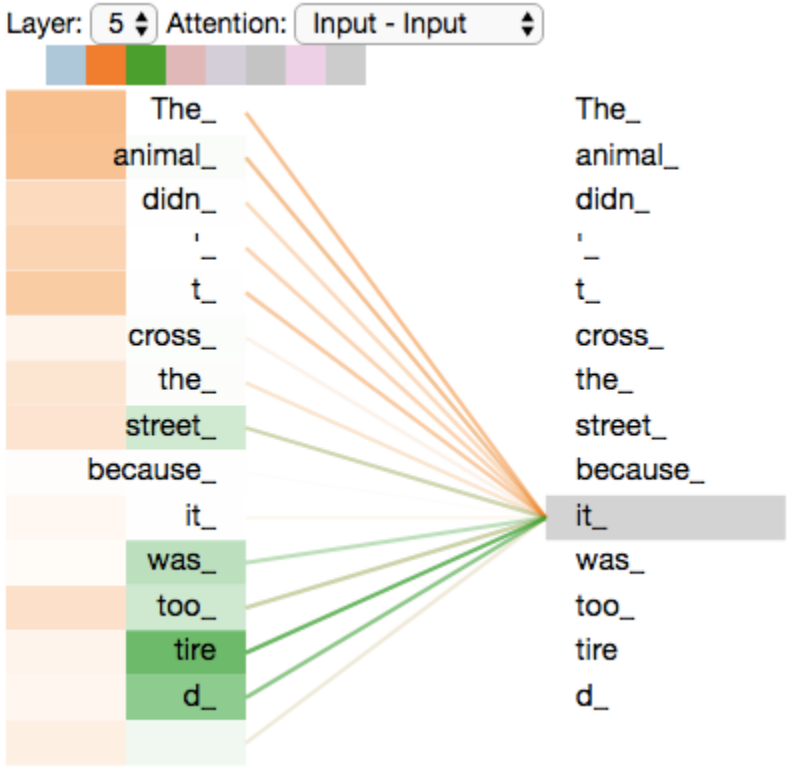


Q (Query) — запрос токена к другим токенам.
K (Key) — ключевые признаки токена для сопоставления.
V (Value) — содержимое токена, которое используется для формирования итогового представления.

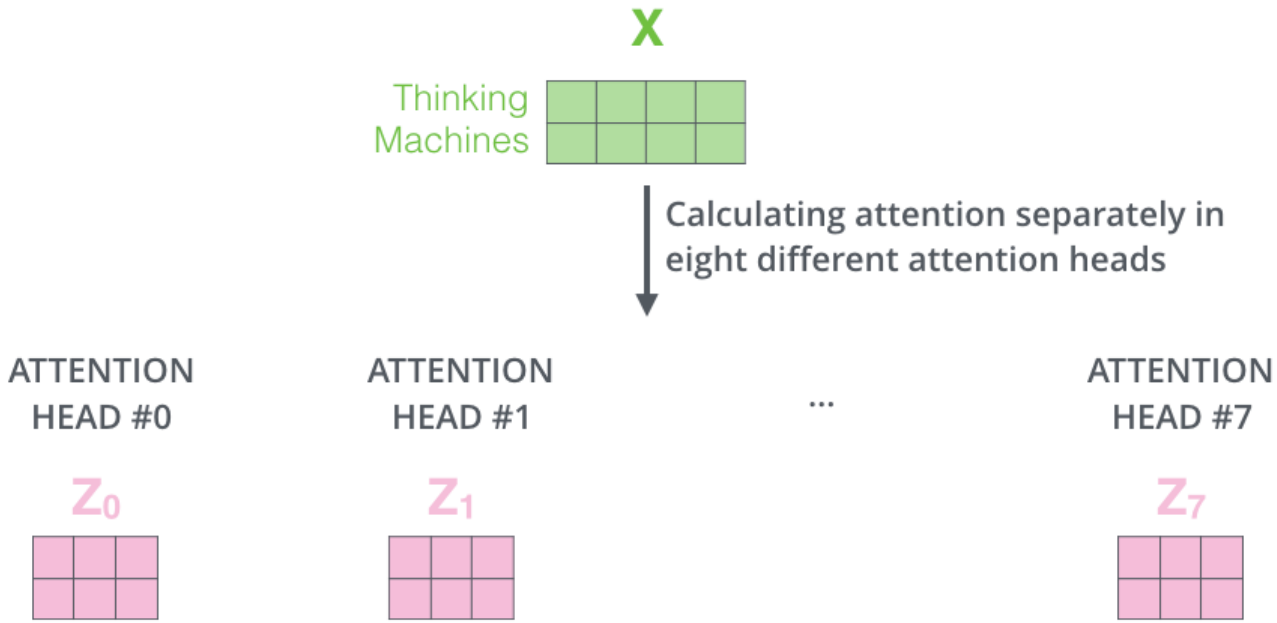
Multi-head Self-Attention



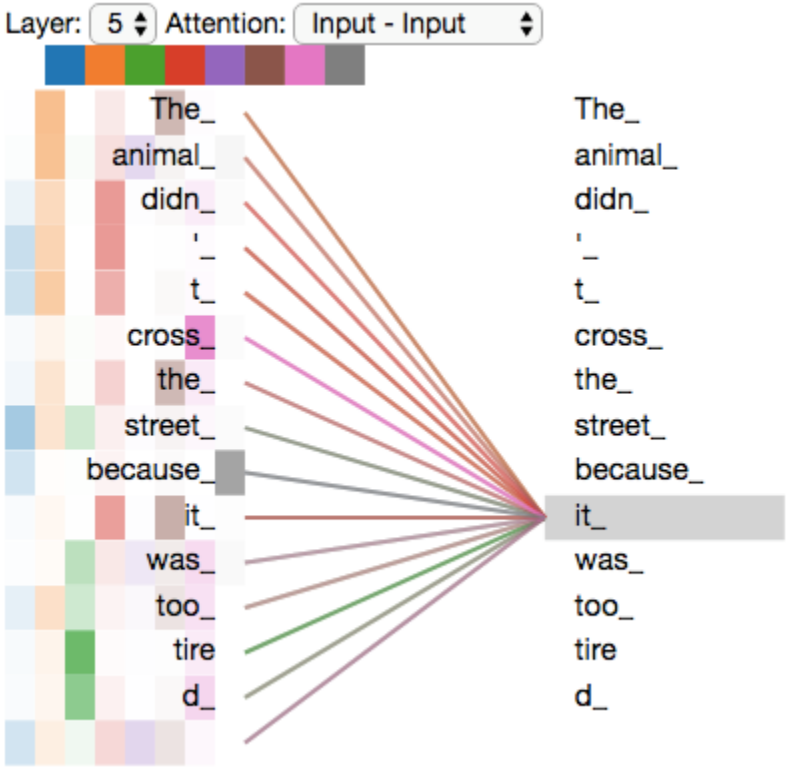
Multi-head Self-Attention



Multi-head Self-Attention



Multi-head Self-Attention

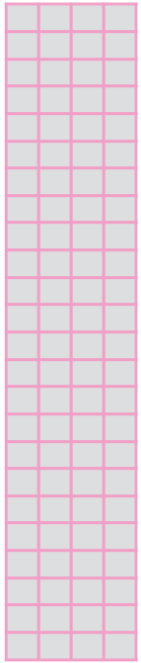


Multi-head Self-Attention

1) Concatenate all the attention heads

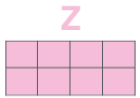
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



Multi-head Self-Attention

1) This is our input sentence*

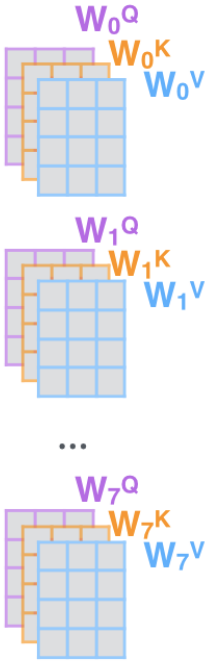
2) We embed each word*

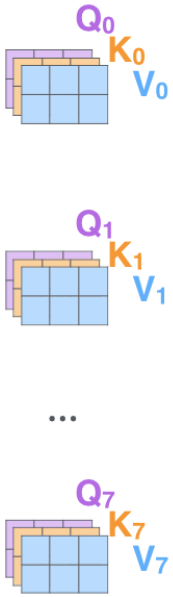
3) Split into 8 heads. We multiply X or R with weight matrices

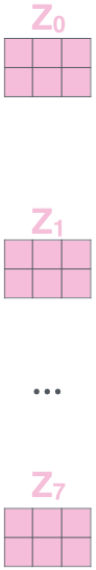
4) Calculate attention using the resulting Q/K/V matrices

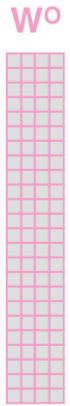
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

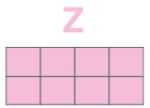
Thinking Machines





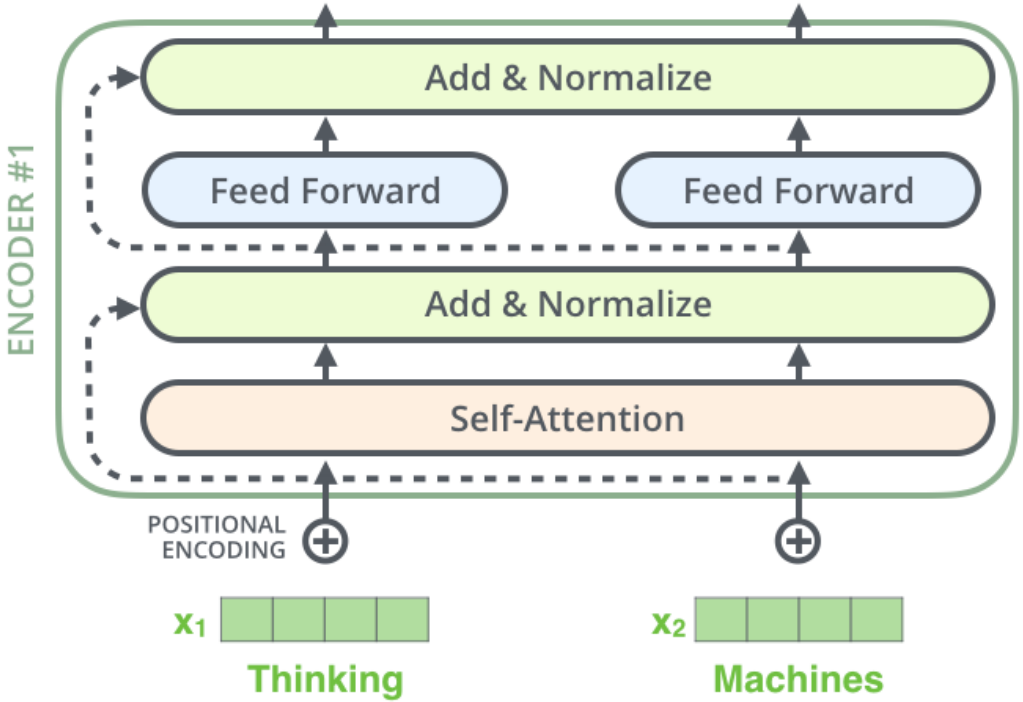




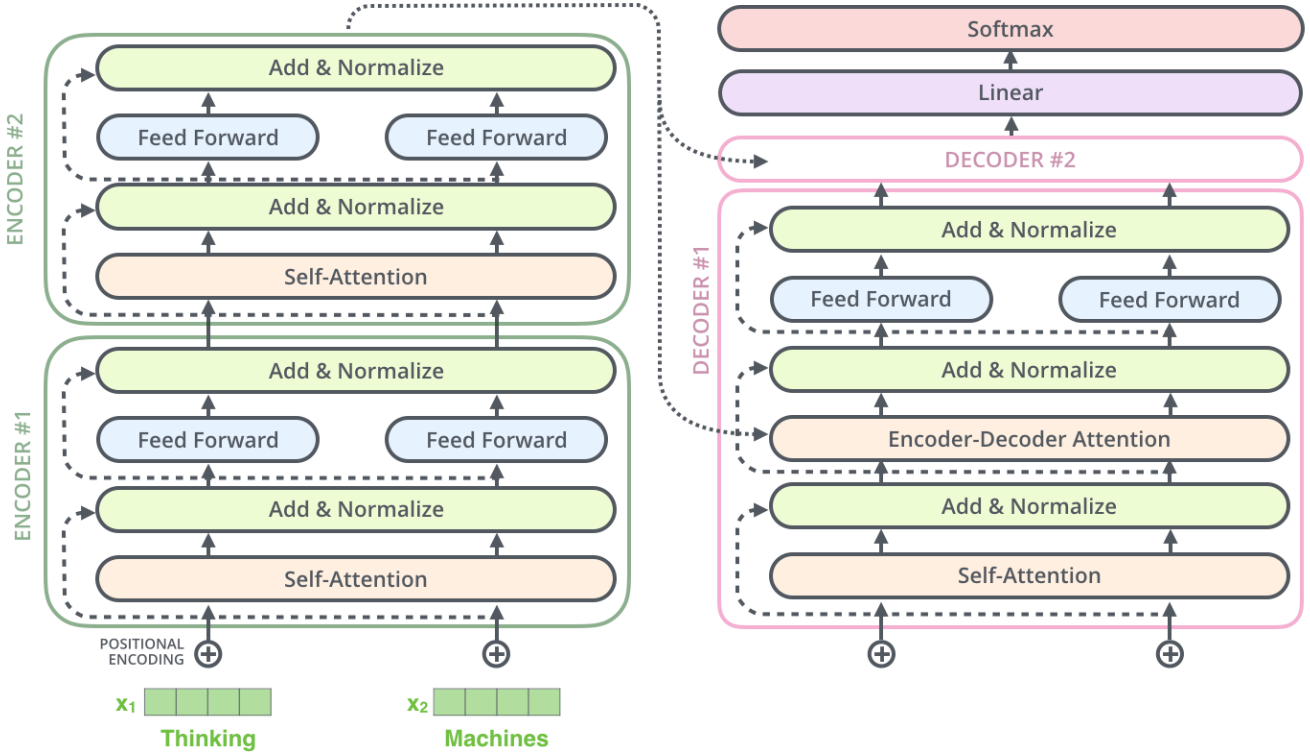


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

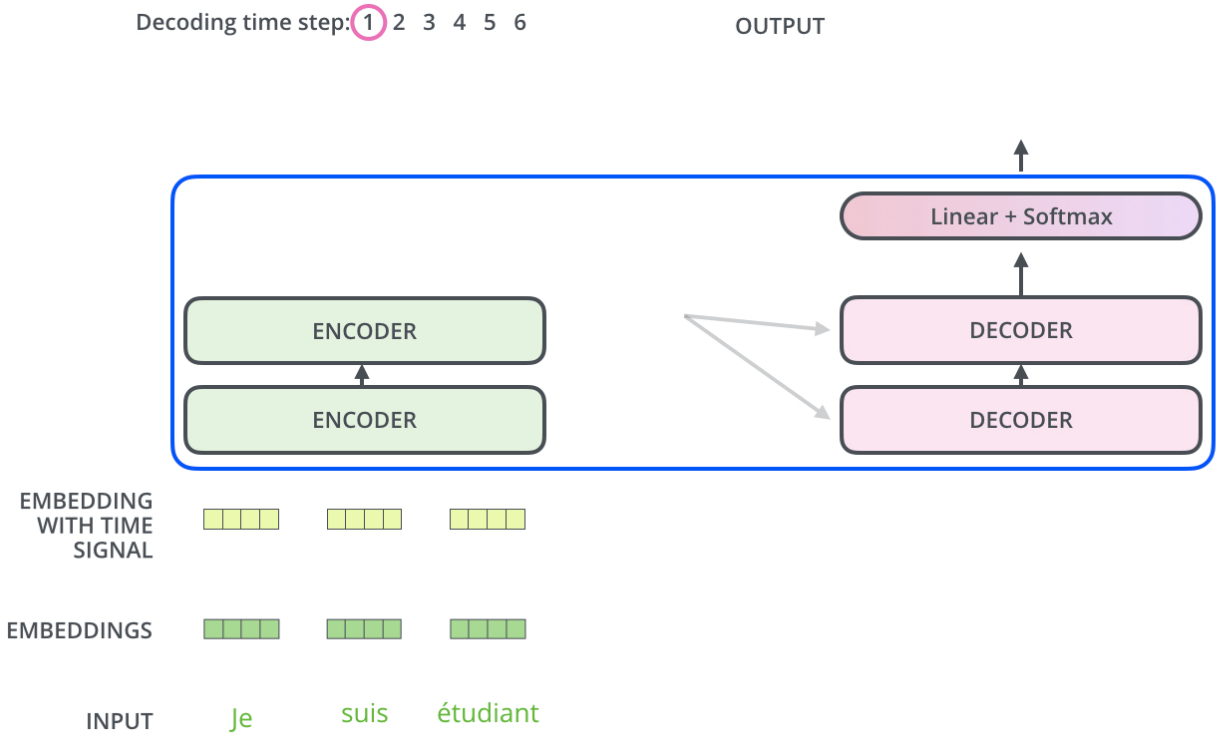
Encoder block



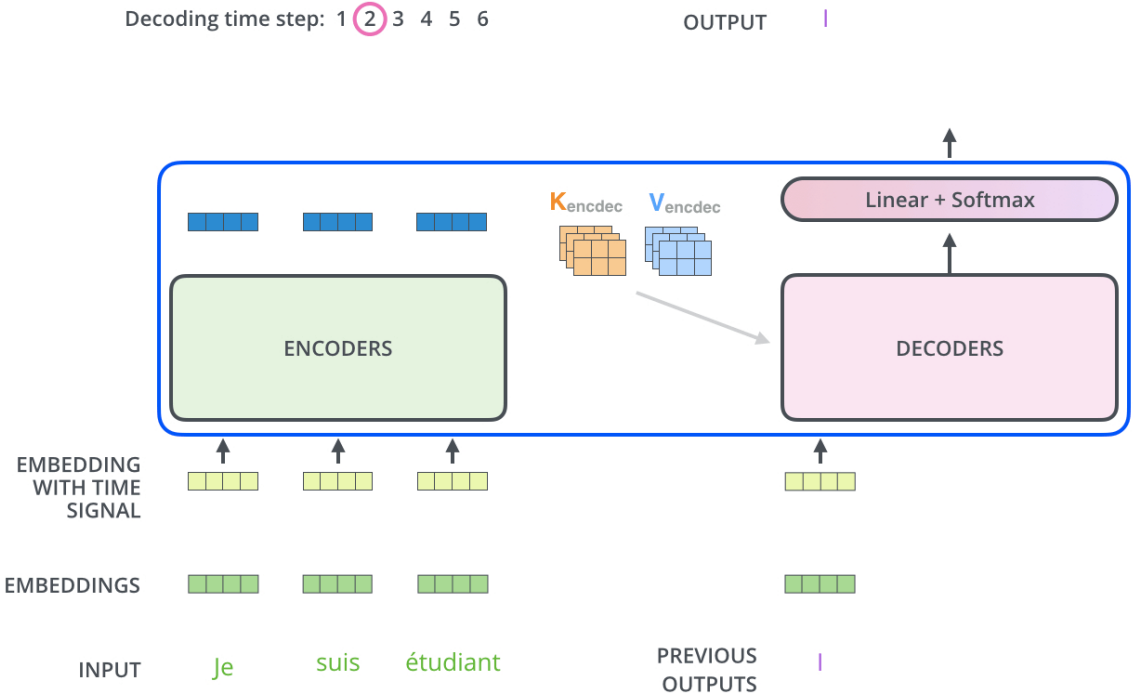
Encoder-Decoder

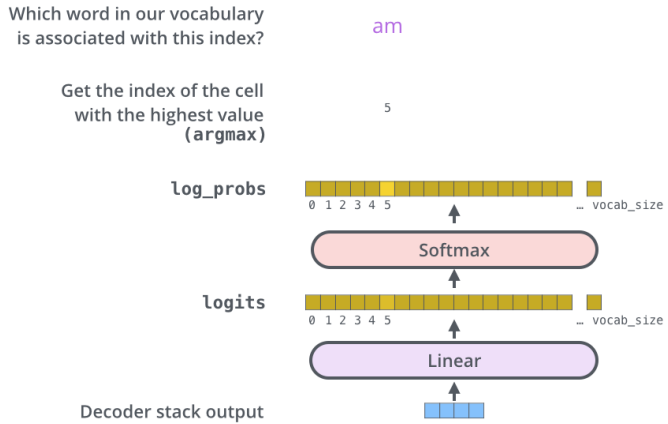


Transformer



Transformer

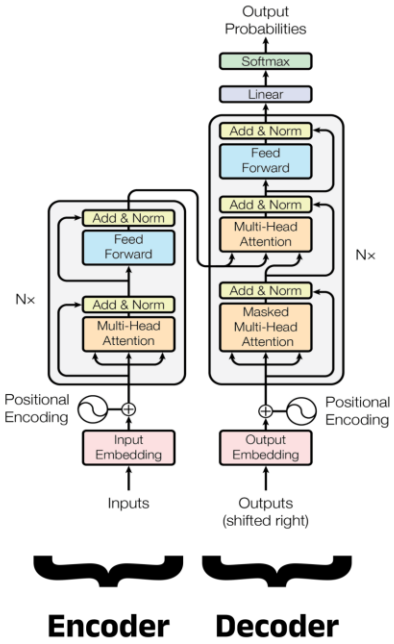




GPT and BERT

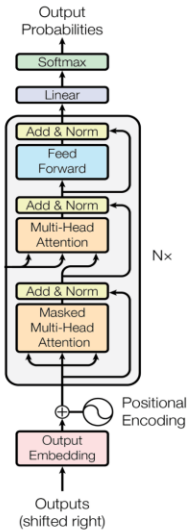
ИТМО

Transformer



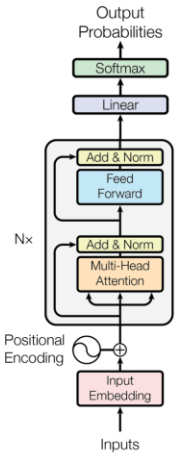
Encoder **Decoder**

GPT*



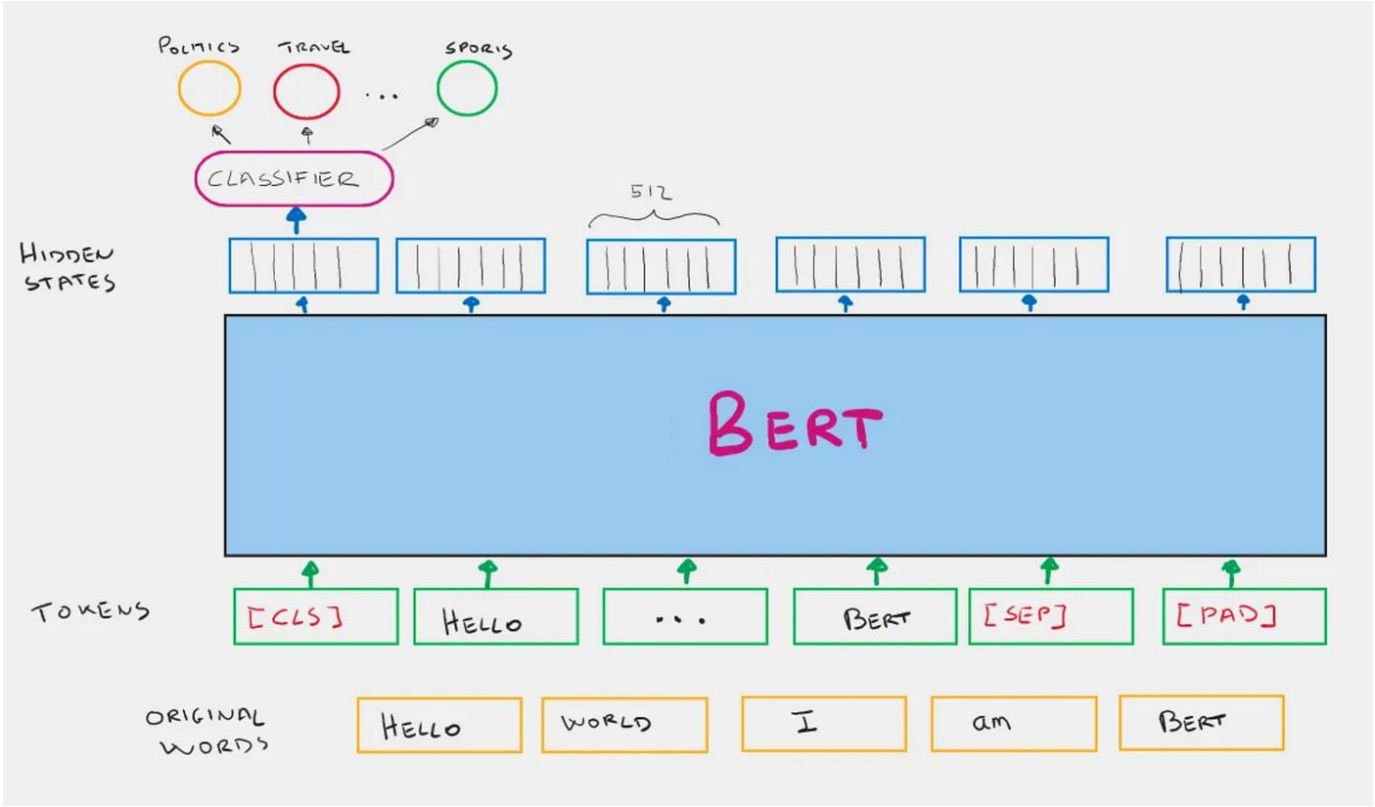
Decoder-only

BERT*



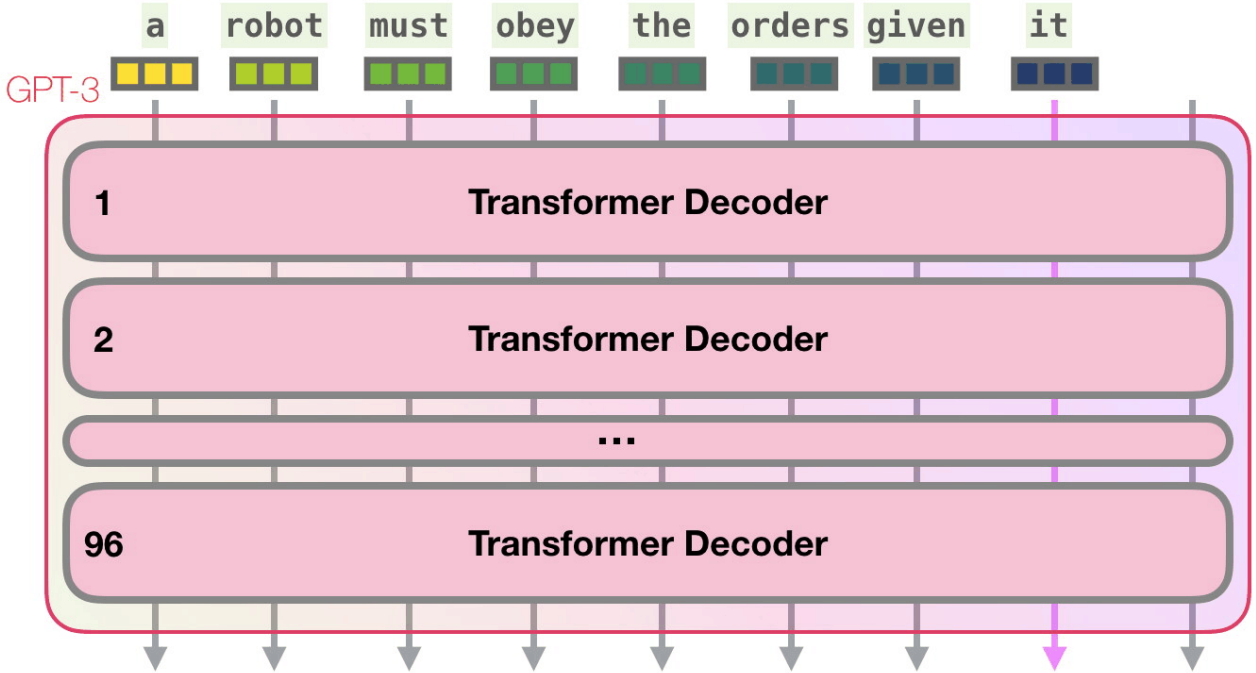
Encoder-only

BERT


POLITICS
TRAVEL
SPORTS
CLASSIFIER
512
HIDDEN STATES
BERT
TOKENS
[CLS]
HELLO
...
BERT
[SEP]
[PAD]
ORIGINAL WORDS
HELLO
WORLD
I
am
BERT

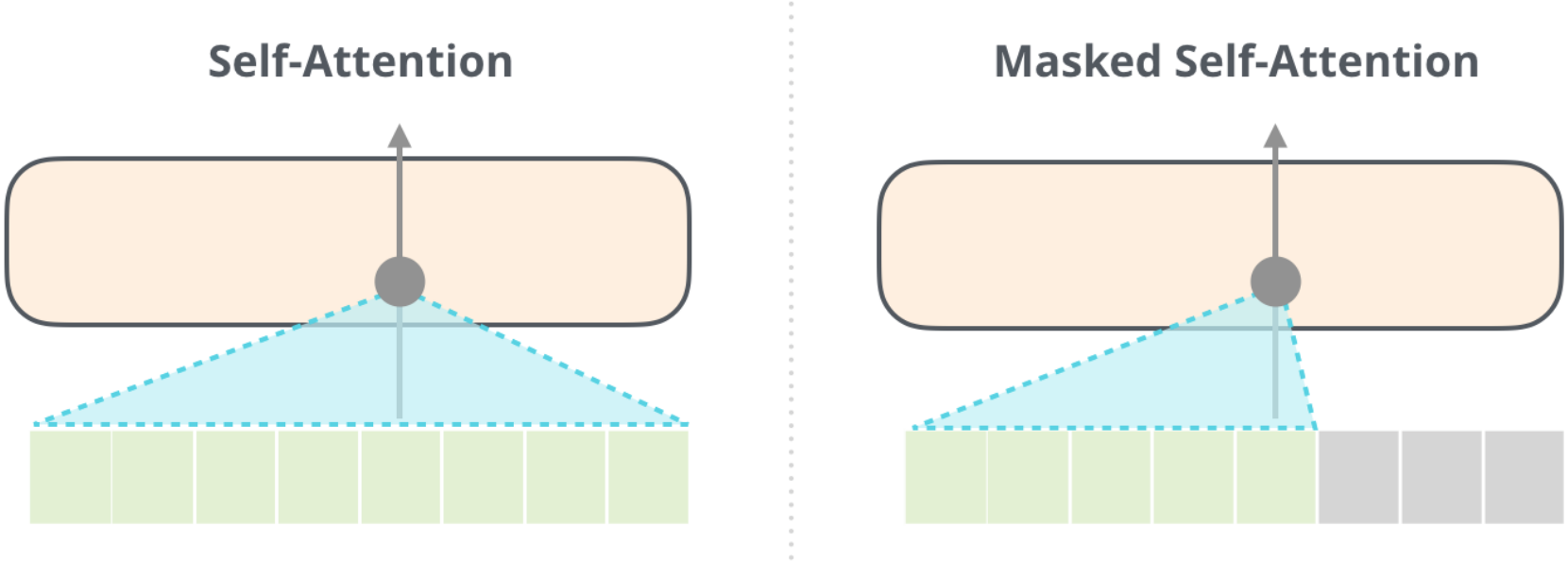
GPT

ИТМО

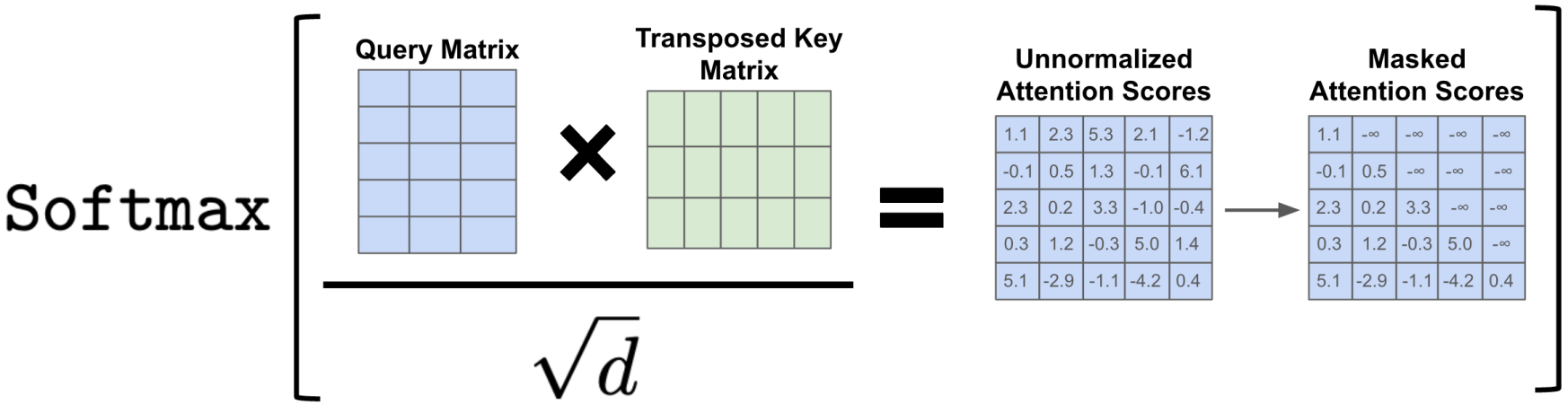

a
robot
must
obey
the
orders
given
it
GPT-3
1
Transformer Decoder
2
Transformer Decoder
...
96
Transformer Decoder

Masked Self-Attention

ИТМО



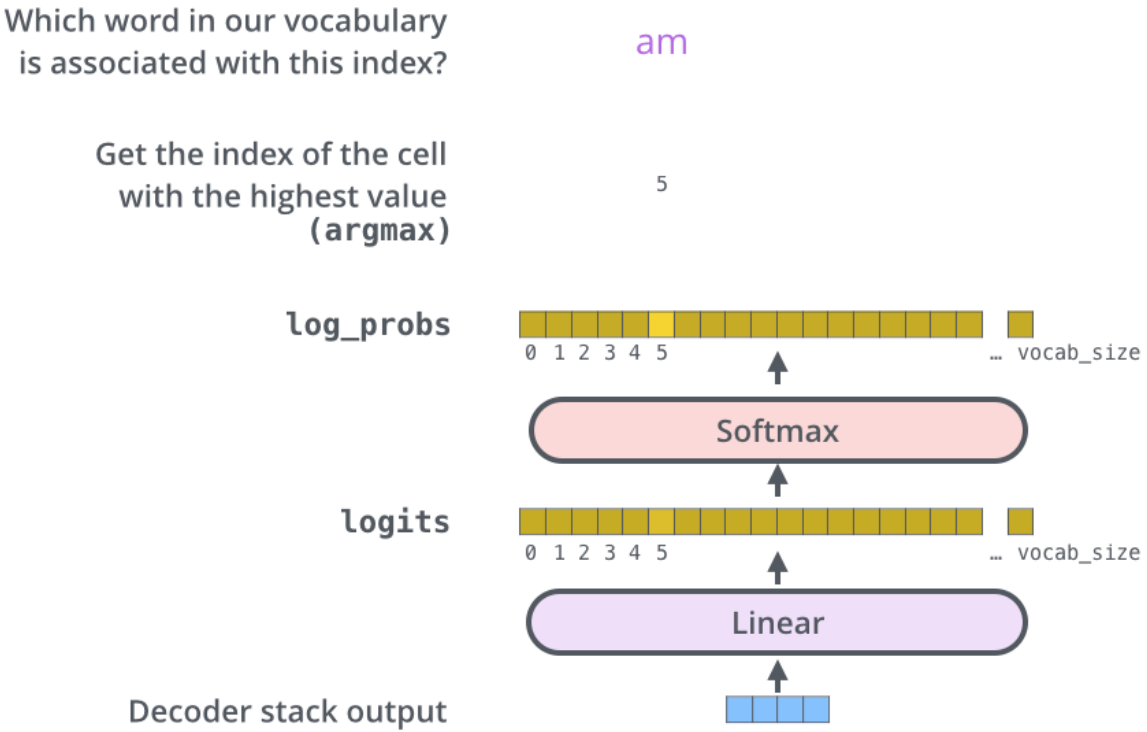
Masked Self-Attention



GPT: визуализация работы

https://bbycroft.net/llm

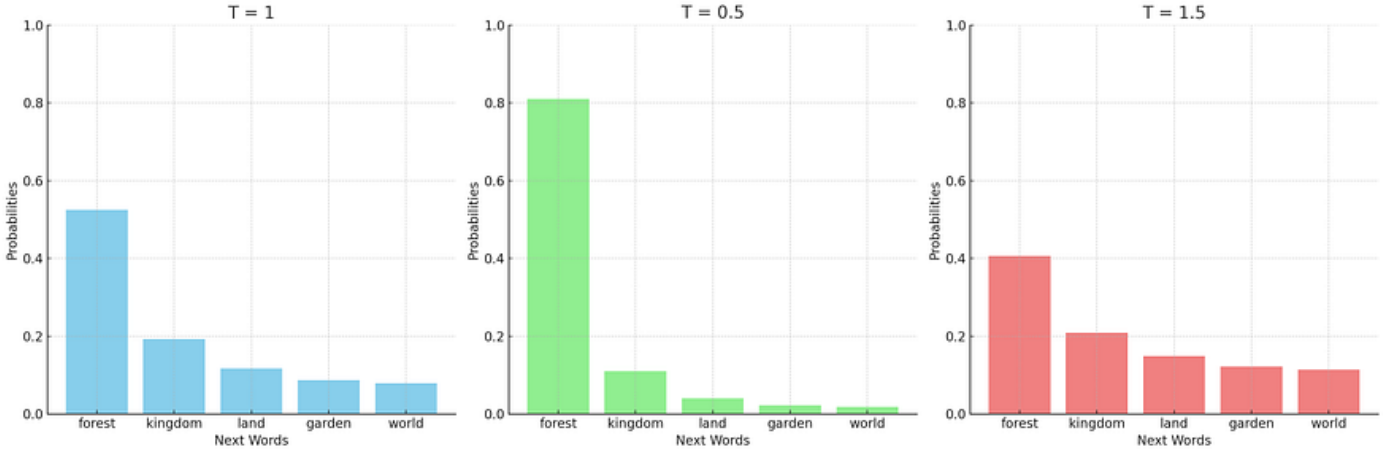
GPT: сэмплирование



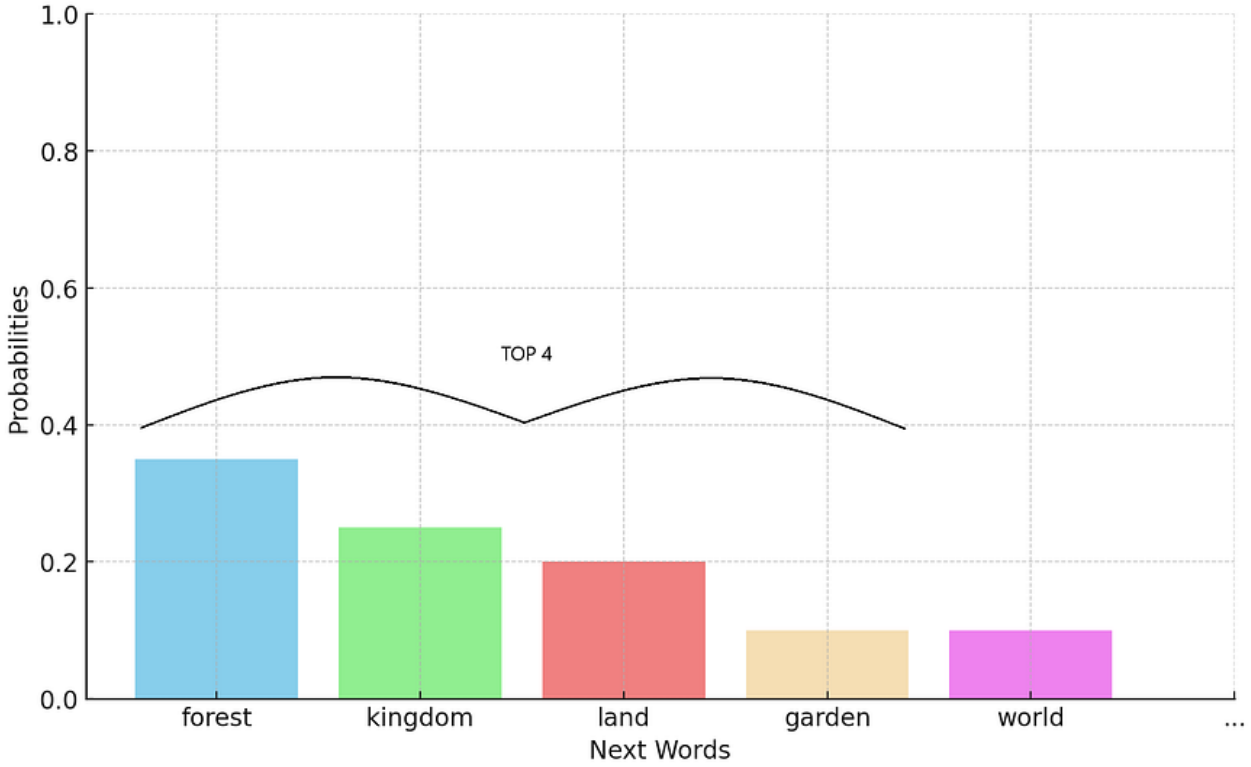
greedy search

Temperature

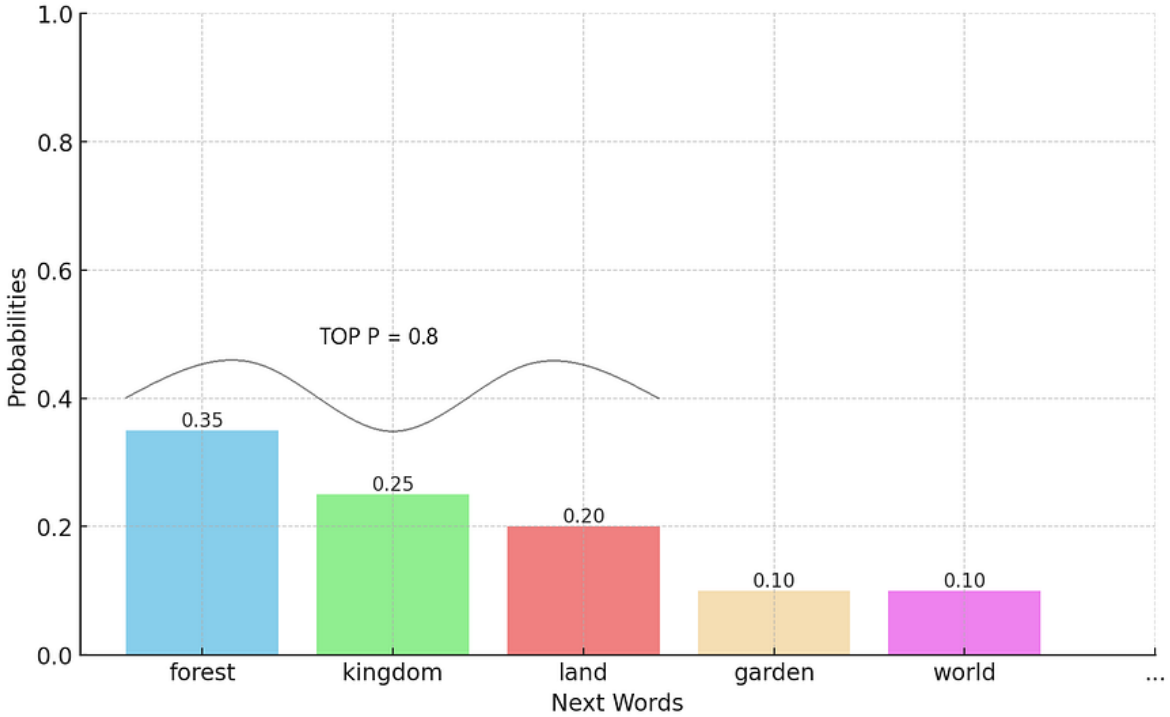
$$\sigma(\mathbf{z})_i = \frac{e^{z_i/T}}{\sum_{j=1}^{K} e^{z_j/T}} \quad \text{for } i = 1, \ldots, K$$



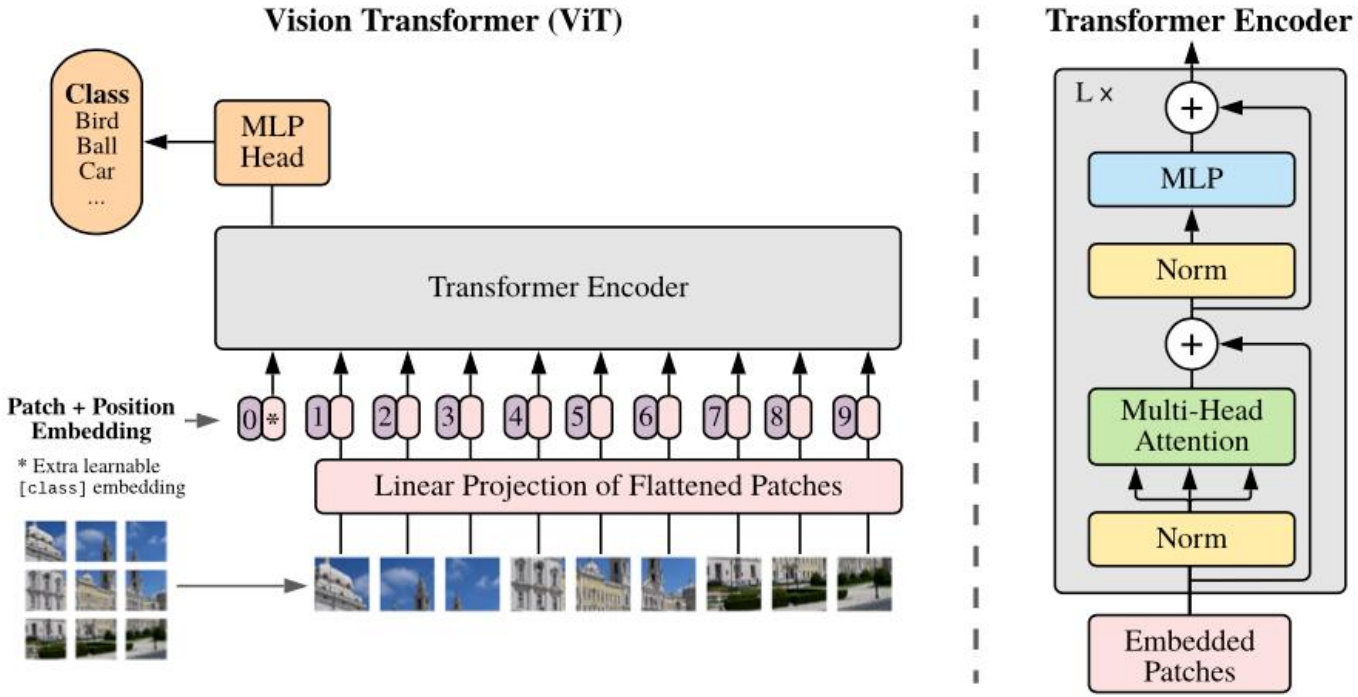
Top-k


1.0
0.8
0.6
0.4
0.2
0.0
Probabilities
TOP 4
forest
kingdom
land
garden
world
...
Next Words

Top-p


1.0
0.8
0.6
0.4
0.2
0.0
Probabilities
TOP P = 0.8
0.35
0.25
0.20
0.10
0.10
forest
kingdom
land
garden
world
...
Next Words

Vision Transformer



Почему Transformer?

Универсальность архитектуры: один и тот же механизм внимания работает и с текстом, и с изображениями, и со звуком.

Эффективный параллелизм: в отличие от других архитектур, трансформеры обучаются быстро за счёт обработки последовательности целиком.

Глубокий контекст: Self-Attention позволяет учитывать зависимости на любых расстояниях, а не только локальные.

Scaling laws: Тема следующей лекции

Спасибо
за внимание!

ITMO more than a UNIVERSITY

Ваши контакты